MunimJi

Interview Preparation & Defense Document

Inventory & GST Billing for Small Shops

For the Managerial / Project Discussion Round

Spring Boot 3.3 · Java 17 · React 18 · Tailwind · JWT · H2/MySQL

13 sections · 140+ Q&A · every claim grounded in this repository

Read the Cheat Sheet (Section 13) the night before. Practice Section 1 speech out loud.

Table of Contents

In Word: right-click the list below and choose **Update Field** to refresh page numbers.

Table of Contents 2

1. Project in 60 Seconds 5

   Elevator pitch 5

   A 2-minute "Tell me about your project" speech (ready to speak) 5

2. Problem and Solution 6

   Problem statement 6

   Target users 6

   What makes it useful 6

   Key features (short list) 6

3. Tech Stack and "Why this?" 7

   Backend 7

   Frontend 7

4. Architecture and Design 8

   Overall architecture: layered monolith 8

   Package structure (real folders in `backend/src/main/java/com/munimji/`) 8

   Design patterns and principles (with real locations) 8

5. How It Works: End-to-End Flows 10

   Flow 1: Login (authentication) 10

   Flow 2: Request authentication/authorization (every API call) 10

   Flow 3: Create bill (the core business flow) 10

   Flow 4: Purchase stock (creates batches) 11

   Flow 5: View PDF invoice (frontend -> backend -> file) 11

   Flow 6: Frontend structure 11

6. Database Design 12

   Tables (9 entities, real columns) 12

   Relationships 12

   Keys, indexes, constraints 12

   Why this schema (normalization + deliberate denormalization) 12

   Transactions and query decisions 13

7. Code Deep Dive 14

   7.1 `InvoiceService.create()` - the heart of the app (backend/src/main/java/com/munimji/service/InvoiceService.java:54) 14

   7.2 `JwtAuthFilter.doFilterInternal()` (security/JwtAuthFilter.java:32) 15

   7.3 Other classes worth quoting 15

7.4 Annotations and config you must be able to explain 15
8. How It Was Built 17
Development order (matches the 25 git commits) 17
Challenges faced and how I solved them (from real code/commits) 17
Git history insights 18
9. Security, Performance and Error Handling 19
Security checklist (what is actually in the code) 19
Performance considerations (what I did / did not do) 19
Error handling architecture 19
10. Testing, Deployment and DevOps 21
Automated tests: 34 in 9 classes (brief asked 15) 21
End-to-end smoke test (the definition-of-done script) 21
How to run / environment configs 21
Deployment and CI/CD - honest status 22
11. Limitations, Improvements and Future Scope 23
Honest weaknesses in the current code 23
How I would scale to 1 lakh / 1 million users 23
What I would change if I rebuilt it 23
12. Interview Q&A Bank 25
a) Project overview and motivation 25
b) Why questions (stack, DB, architecture, approaches) 26
c) How questions (flows and mechanisms) 28
d) Code-level questions (specific classes, methods, annotations) 30
e) Database questions (schema, joins, transactions, indexing) 33
f) API and backend design questions 35
g) Frontend questions 36
h) Security questions 38
i) Scalability and performance questions 39
j) Challenges, bugs, failures and learnings 40
k) Managerial and behavioral questions tied to the project 42
l) Tricky / cross questions (and handling "I don't know") 44
m) Scenario questions (what-if) 45
13. Cheat Sheet - One-Page Rapid Revision 48
Stack in one line each 48
Five flows in one line each 48
Key numbers (memorize) 49
Top 16 questions to rehearse 49

Top 10 things never to say 49

- **How to use this document**: Sections 1-2 are your opening answers; 3-7 are depth for technical rounds; 8-11 handle experience, trade-offs and future scope; Section 12 is the drill bank; Section 13 is last-night revision.
- Every code reference (class names, methods, file paths) is taken from the actual repository - open the file if asked.
- Anything the repo does not contain is marked **"Not found in repo"** (for example: CI/CD, Docker, frontend tests).

1. Project in 60 Seconds

Elevator pitch

- **MunimJi** is a **full-stack inventory + GST billing web app** for small shops (kirana stores, pharmacies).
- A shopkeeper buys stock (batches with expiry dates), sells it at the counter with **GST invoices**, and always sees low-stock and near-expiry alerts.
- Built alone as a **portfolio project**: **Spring Boot 3.3 + Java 17** API, **React 18 + Tailwind** UI, **JWT security**, **H2/MySQL, multi-tenant shop isolation**, **34 automated tests** and a **62-check end-to-end smoke test**.
- The interesting part: **FEFO stock deduction** (First-Expiry-First-Out) with GST math and **atomic rollback** in one transaction.

A 2-minute "Tell me about your project" speech (ready to speak)

```
"MunimJi is an inventory and GST billing application I built for small
 shops like kirana stores. The problem is real: shopkeepers lose money
 when stock expires on the shelf, and making GST-compliant bills by hand
 is slow and error-prone.

 It has two sides. The backend is Spring Boot 3.3 with Java 17,
 following a clean layered architecture - controllers handle HTTP,
 services hold the business rules, repositories talk to the database.
 Security is stateless JWT with two roles: OWNER and STAFF. The
 frontend is React with Tailwind CSS - a POS-style billing screen,
 product and supplier management, stock alerts, and reports with charts.

 The core feature I am most proud of is billing. When a bill is saved,
 four things happen inside ONE database transaction: I calculate GST
 per line with CGST/SGST split, distribute any invoice-level discount
 across lines proportionally, deduct stock from batches using FEFO -
 first expiry first out - and assign an invoice number like
 INV-2026-0001. If any product does not have enough stock, the whole
 transaction rolls back - no invoice, no partial stock deduction.

 For quality I wrote 34 JUnit and MockMvc tests, plus a 62-check
 end-to-end smoke script that proved FEFO ordering, GST amounts,
 rollback, role-based access and PDF invoice generation. Everything is
 documented in a README and interview notes. The stack choice was
 deliberate: H2 so anyone can run it with zero setup, MySQL supported
 through environment variables for production."
```

> **Interview Tip:** Speak the speech out loud twice before the interview. Keep the 60-second version for 'tell me briefly' and the 2-minute one for the full ask.

2. Problem and Solution

Problem statement

- Small shops manage stock on paper or in Excel - they **lose money when products expire** because nothing warns them.
- Manual GST bills are slow, add up wrongly, and are hard to file - CGST/SGST split and HSN codes are confusing for shopkeepers.
- Existing software is either **too big/expensive** (Tally, enterprise ERPs) or **too simple** (no batch/expiry concept).
- No free, modern, browser-based tool that is simple enough for a kirana shop owner with a 10-minute setup.

Target users

- **Owner** - manages products, suppliers, purchases, sees reports and GST summary.
- **Staff (billing clerk)** - can only search products, view stock, and create/view invoices at the counter. (Enforced in code: `SecurityConfig.java`.)
- Shop type: small retail with batch/expiry-sensitive goods (grocery, pharmacy).

What makes it useful

- Bill in seconds with **FEFO** - the system always sells the batch that expires first, so less wastage.
- GST is calculated automatically per line (CGST + SGST) and printed on a **downloadable PDF invoice**.
- **Alerts before money is lost**: low stock vs reorder level, batches expiring in N days, already-expired batches.
- One-click GST summary per month for filing, plus sales/top-product charts.

Key features (short list)

- **Billing / POS**: cart, invoice-level discount, GST per line, FEFO deduction, invoice number INV-YYYY-NNNN, PDF download.
- **Inventory**: products with HSN, GST rate, barcode, reorder level; suppliers; purchase entry that creates stock batches.
- **Stock view + alerts**: total stock per product, low stock, expiring (default 45 days), expired.
- **Reports**: today/month sales, monthly sales line chart, top products, GST summary with rate breakdown.
- **Auth**: JWT login, OWNER vs STAFF roles, public shop registration (anonymous sign-up creates an OWNER; an OWNER adds STAFF).
- **Docs**: README, decisions log, 34 tests, 62-check smoke test script.

> **Remember:** Anchor every feature to money: FEFO saves expired stock, GST accuracy avoids filing mistakes, alerts prevent stockouts. Managers think in money, not code.

3. Tech Stack and "Why this?"

Backend

- **Java 17 (LTS)** - long-term support, records/switch expressions, safest production choice. Alternative: Java 21 (fine, but 17 is the classic LTS asked in interviews).
- **Spring Boot 3.3.5** - one dependency brings embedded Tomcat, auto-configuration, externalized config. Alternative: Jakarta EE / Micronaut / Quarkus - Spring is the most in-demand.
- **Maven** - standard build tool, dependency resolution, one `mvn clean test` for CI. Alternative: Gradle (faster but less common in jobs).
- **Spring Data JPA (Hibernate)** - repositories with almost zero SQL; I wrote custom `@Query` JPQL only where it mattered (FEFO, JOIN FETCH). Alternative: plain JDBC or MyBatis - too much boilerplate for this scale.
- **H2 (default) + MySQL driver** - H2 file DB so anyone runs it with **zero install**; same code switches to MySQL via 4 env vars. Alternative: MySQL-only (kills the 'clone and run' experience), Postgres (not required).
- **Spring Security + jjwt 0.11.5** - stateless JWT (no server sessions - ready to scale horizontally). Alternative: session cookies (simpler, but ties you to one server), OAuth2 providers (overkill for a shop app).
- **OpenPDF 1.3.43** - lightweight, Apache-licensed, pure Java PDF generation. Alternative: iText (license cost), wkhtmltopdf (external binary).
- **spring-boot-starter-validation (Jakarta Bean Validation)** - declarative `@NotBlank`, `@DecimalMin` on DTOs instead of manual if-checks.
- **No Lombok** - plain getters/setters so interviewers can read every line (and it avoids build plugins).

Frontend

- **React 18** - component model fits a dashboard with shared Modal/Loader/Sidebar. Alternative: Angular (heavier for a solo project), Vue (similar, React has more jobs).
- **Vite 5** - instant dev server, fast production build (built in ~50 s). Alternative: Create React App (deprecated, slow).
- **Tailwind CSS 3.4** - custom design system (ink/brand/gold palette, `.btn`, `.card` classes) without leaving the markup. Alternative: Bootstrap (generic look), Material UI (opinionated look).
- **React Router 6** - client-side routes with a `ProtectedRoute` wrapper. **Redux not used** - one `AuthContext` is enough for a small app (and I can defend that choice).
- **Axios** - interceptors add the JWT header and auto-logout on 401 in one place (`src/api/axios.js`).
- **Recharts** - declarative charts for monthly sales / top products; code-split into its own bundle chunk in `vite.config.js`.
- **lucide-react** - consistent icon set for the UI.

> **Interview Tip:** Never say "it was in the tutorial". Say: "I picked X because of Y constraint - and the trade-off was Z." That one sentence pattern wins stack questions.

4. Architecture and Design

Overall architecture: layered monolith

- **Monolith, layered (n-tier)** - one deployable Spring Boot jar serving a REST API; the React app is a separate SPA that talks to it over HTTP.
- Request travels: **Controller -> Service -> Repository -> Database**, then DTO comes back up. Controllers never touch repositories.
- Stateless security: no HTTP sessions, JWT on every request.

```
Browser (React SPA, localhost:5173)
   |  Axios: Authorization: Bearer <jwt>
   v
[ JwtAuthFilter ]--> validates token, fills SecurityContext
   |
   v
[ @RestController ]   --- DTO in / DTO out ---     (com.munimji.controller)
   |
   v
[ @Service ]          --- business rules, @Transactional, FEFO, GST ---
(com.munimji.service)
   |
   v
[ JpaRepository ]     --- JPQL / SQL ---           (com.munimji.repository)
   |
   v
[ H2 (dev) or MySQL (prod) ]  tables: product, stock_batch, invoice, ...
```

Package structure (real folders in `backend/src/main/java/com/munimji/`)

```
config/        SecurityConfig - filter chain, CORS, BCrypt beans
controller/    8 controllers: Auth, Category, Product, Supplier,
               Purchase, Stock, Invoice, Report (PDF endpoint in InvoiceController)
service/       Auth, Category, Product, Supplier, Purchase, Stock,
               Invoice, Report, Pdf services (all business logic)
repository/    9 Spring Data JPA interfaces with custom @Query
entity/        9 entities: User, Category, Product, Supplier,
               Purchase, PurchaseItem, StockBatch, Invoice, InvoiceItem
dto/           request/response objects grouped by feature
               (dto/invoice, dto/product, dto/auth, ... PagedResponse)
security/      JwtUtil, JwtAuthFilter, UserDetailsServiceImpl, AppUserDetails, CurrentTenant
exception/     ApiException + 6 subclasses + GlobalExceptionHandler
util/          MoneyUtil (BigDecimal rounding), AmountInWordsUtil

frontend/src/  api/axios.js  context/AuthContext.jsx
               components/  (Layout, Sidebar, Navbar, Modal, ...)
               pages/       (Login, Register, Dashboard, Billing, Products, ... Users)
```

Design patterns and principles (with real locations)

- **Layered architecture** - separation of concerns: `ProductController.list()` only maps HTTP; `ProductService` holds rules; `ProductRepository` holds queries.
- **DTO pattern** - entities never leave the API. `InvoiceRequest` in, `InvoiceResponse` out. Stops JSON cycles and hides DB details.
- **Repository pattern** - Spring Data interfaces; custom FEFO query lives in `StockBatchRepository.findSellableByProduct()`.
- **Query scoping** - every repository method takes the tenant id (`search(ownerId, ...)`, `findTopByOwnerIdOrderByIdDesc`); isolation lives in the query signatures, not in call-site discipline.
- **Central error handling** - `@RestControllerAdvice GlobalExceptionHandler` turns every exception into `{status, message}`.

- **Composite** - `Invoice` has `List<InvoiceItem>` with `cascade = ALL`: save one aggregate, children save automatically.
- **Template/strategy light** - `OncePerRequestFilter` subclass `JwtAuthFilter` plugs into Spring's filter chain.
- **Principles**: single responsibility (one service = one domain), DRY (`MoneyUtil.round()` used everywhere), fail-fast (throw `BadRequestException` early), constructor injection everywhere (no field `@Autowired`, easier tests).

> **Note:** I deliberately avoided patterns a fresher would misuse: no factory/singleton by hand, no repository-per-DTO explosion. The code stays readable - that is a feature, not a gap.

5. How It Works: End-to-End Flows

Flow 1: Login (authentication)

```
React Login.jsx  --POST /api/auth/login {email,password}-->
AuthController.login --> AuthService.login()
   -> AuthenticationManager.authenticate()   (Spring Security verifies BCrypt hash)
   -> UserRepository.findByEmail()
   -> JwtUtil.generateToken(user)            (HS256, claims: role+name, sub=email, 24h)
   <-- 200 { token, name, email, role }
AuthContext.login() stores token+user in localStorage
Every later call: axios request interceptor adds "Authorization: Bearer <token>"
```

- Wrong password -> `authenticationManager` throws -> caught in `AuthService.login()` and re-thrown as `UnauthorizedException` -> `{"status":401,"message":"Invalid email or password"}` (no leak of which field was wrong).
- Frontend `ProtectedRoute` checks `token` in context; missing -> redirect to `/login`.

> **Remember:** Password never travels back: DB stores a **BCrypt hash** (salted, one-way). `BCryptPasswordEncoder` bean in `SecurityConfig`.

Flow 2: Request authentication/authorization (every API call)

```
HTTP request with Bearer token
 -> JwtAuthFilter.doFilterInternal()          (runs before
UsernamePasswordAuthenticationFilter)
    -> JwtUtil.extractEmail(token)            (verifies signature with secret)
    -> JwtUtil.isValid(token, email)          (signature + subject + not expired)
    -> UserDetailsServiceImpl.loadUserByUsername(email)
    -> SecurityContextHolder.getContext().setAuthentication(...)   // user is now "logged in"
 -> SecurityConfig URL rules decide:
    /api/auth/login, /register        -> permitAll
    GET /api/products|categories|stock/**, GET+POST /api/invoices -> OWNER or STAFF
    /api/reports/**                   -> OWNER only
    everything else (POST/PUT/DELETE)  -> OWNER only
 -> 401 if not authenticated, 403 if wrong role (custom JSON bodies)
```

- Bad/expired token: the filter catches the exception and continues **without** a user - the URL rules then answer 401. One place, consistent behavior.
- STAFF hitting `POST /api/products` gets **403** - proven in tests `SecurityRulesTest` (5 tests).

Flow 3: Create bill (the core business flow)

```
Billing.jsx builds cart {items:[{productId,quantity}], discount}
 --POST /api/invoices--> InvoiceController.create
 -> InvoiceService.create()   @Transactional   ONE transaction:
    1. validate discount >= 0 and <= subtotal          (BadRequestException)
    2. load each Product, gross = sellingPrice * qty   (MoneyUtil.round)
    3. split discount proportionally per line;
       LAST LINE GETS THE REMAINDER so parts add up exactly
    4. per line: taxable = gross - share
                 gst = taxable * gstRate / 100          (scale 8, HALF_UP)
                 cgst = gst / 2 ; sgst = gst - cgst
    5. FEFO: StockBatchRepository.findSellableByProduct()
             -> qty > 0, expiry >= today, ORDER BY expiry ASC, id ASC
       sum available; if < needed -> InsufficientStockException
       deduct: take = min(batch.qty, remaining) loop
    6. invoiceNumber = nextInvoiceNumber()  (INV-2026-0001, reads last invoice)
    7. build Invoice + InvoiceItems (SNAPSHOT name/hsn/price/gstRate)
    8. cascade save (Invoice + items in one insert set)
 <-- 200 InvoiceResponse
Any exception -> automatic ROLLBACK: no invoice, no stock deducted
```

Interview Tip: Say the four magic words in one breath: "GST, discount split, FEFO, all in ONE transaction with rollback." That sentence defines the whole project.

Flow 4: Purchase stock (creates batches)

```
PurchaseNew.jsx --POST /api/purchases {supplierId, items[]}--> PurchaseService.create()
 -> supplier must exist (else 404 ResourceNotFoundException)
 -> for each line: save PurchaseItem AND create one StockBatch
       (product, batchNo, expiryDate, quantity, costPrice)
 -> purchase.total = sum(costPrice * qty)
 -> cascade save in one @Transactional block
Stock page then reads stock_batch via StockService
```

Flow 5: View PDF invoice (frontend -> backend -> file)

```
Invoices.jsx "Download PDF" button
 -> axios GET /api/invoices/{id}/pdf   (with Authorization header)
 -> InvoiceController -> InvoiceService.findById (with items)
 -> PdfService.generateInvoicePdf(response)
       lowagie Document + PdfWriter -> ByteArrayOutputStream
       shop name/GSTIN/address from config, item table,
       totals, "amount in words" (AmountInWordsUtil), footer
 -> 200 with Content-Type: application/pdf
 -> browser converts the blob to a download
```

- It is a **blob fetch**, not a plain `<a href>`, because the PDF endpoint is **protected** and a plain link cannot send the JWT header.
- Verified by smoke check: response starts with `%PDF` and ends with `%%EOF`.

Flow 6: Frontend structure

- `App.jsx` routes: `/login` and `/register` public; everything else wraps `ProtectedRoute` -> `Layout` (sidebar + navbar + page outlet).
- `AuthContext` holds `{token, user, isOwner, login, logout}` - `isOwner` hides owner-only buttons (UI hint only; **real security is on the server**).
- Axios response interceptor: any **401** clears localStorage and redirects to `/login` - one global logout.
- Design system lives in `src/index.css` (`.card`, `.btn`, `.input`) + `tailwind.config.js` (colors, fonts, animations).

6. Database Design

Tables (9 entities, real columns)

- **users** - id, name, email (UNIQUE), password (BCrypt), role (OWNER/STAFF, `@Enumerated(STRING)`), owner_id (null for OWNER, the owner's user id for STAFF).
- **category** - id, name, owner_id (name unique per shop, checked in the service).
- **product** - id, name, barcode, unit, hsn_code, gst_rate (5,2), selling_price (19,2), reorder_level, category_id FK (LAZY), owner_id.
- **supplier** - id, name, phone, email, address, gstin, owner_id.
- **purchase** - id, supplier_bill_no, supplier_id FK, purchase_date, total (19,2), owner_id.
- **purchase_item** - id, purchase_id FK, product_id FK, quantity, cost_price, batch_no, expiry_date.
- **stock_batch** - id, product_id FK, batch_no, expiry_date, quantity, cost_price, **version** (`@Version` optimistic lock).
- **invoice** - id, invoice_number (unique per shop), invoice_date, customer_name, customer_phone, sub_total, discount_amount, cgst_amount, sgst_amount, gst_amount, grand_total, owner_id - money columns (19,2).
- **invoice_item** - id, invoice_id FK, product_id FK, stock_batch_id FK + SNAPSHOT columns: product_name, hsn_code, unit_price, gst_rate, quantity, discount_amount, taxable_amount, gst_amount, line_total.

Relationships

- Category 1-N Product; Supplier 1-N Purchase; Product 1-N StockBatch; Product 1-N PurchaseItem.
- Invoice 1-N InvoiceItem (`cascade = ALL, orphanRemoval = true` - delete invoice, items die with it).
- InvoiceItem N-1 Invoice / Product / StockBatch (all `FetchType.LAZY`, `invoice_id` is `optional = false`).
- Diagram: `category --< product --< stock_batch`, `product --< invoice_item >-- invoice`, `supplier --< purchase --< purchase_item`.

Keys, indexes, constraints

- Primary keys: `@GeneratedValue(strategy = IDENTITY)` on every entity.
- Unique: `users.email` via `@Column(unique = true)` - DB level. The shop-scoped ones are per-tenant: `category.name`, barcode and invoice numbers are checked only inside the caller's own rows (duplicates return 409).
- Money columns: `precision = 19, scale = 2`; GST rate `precision = 5, scale = 2` - matches `DECIMAL` in DB.
- **No hand-written index DDL in the repo** - schema comes from Hibernate (`ddl-auto=update`). Honest answer: for production MySQL I would add indexes on `stock_batch(product_id, expiry_date)`, `invoice(invoice_date)`, (owner_id, ...) composites for tenant scans and FK columns.

Why this schema (normalization + deliberate denormalization)

- Normalized to about **3NF** - prices/tax live on product, transactions in invoice tables, no repeated supplier details in invoices.
- **Deliberate denormalization in invoice_item**: product_name, hsn_code, unit_price, gst_rate are **snapshots**. If the shop renames a product or changes GST next year, **old bills and PDFs stay correct**. This is the one place I chose history over purity.
- Splitting stock into **stock_batch** rows (instead of one quantity on product) is what makes FEFO and expiry alerts possible.

- GST totals stored on both invoice (fast list/summary) and item (per-line PDF) - small duplication for simple reports.

Transactions and query decisions

- `@Transactional` on service write methods (`InvoiceService.create`, `PurchaseService.create`); `@Transactional(readOnly = true)` on reads - a hint to the ORM/driver to optimize.
- All-or-nothing: any exception inside `create()` rolls back invoice **and** stock deduction (proven by smoke test rollback check).
- Concurrency: `@Version` on StockBatch -> optimistic locking; simultaneous sells of the last unit -> second gets 409 "Stock was updated by someone else. Please retry." (handled in `GlobalExceptionHandler`).
- Avoiding **N+1**: `JOIN FETCH` in `findByIdWithItems`, `findAllWithDetails`, `findAllWithBatches`, plus `DISTINCT` where a collection join multiplies rows.
- Reports use **Java loops over queried invoices** instead of SQL `GROUP BY` - portable across H2/MySQL, easy to read; for big data I would move to aggregate SQL (see section 11).

> **Common Mistake:** Do not claim "I added indexes on X" if there is no index DDL in the repo. Interviewers may ask to see it. Say what exists, then say what you WOULD add - that is stronger.

7. Code Deep Dive

7.1 `InvoiceService.create()` - the heart of the app (backend/src/main/java/com/munimji/service/InvoiceService.java:54)

Four jobs in one method: validate, price, deduct stock, save. It is annotated `@Transactional`, so all of it is one atomic unit.

Discount split (proportional, remainder on last line)

```
for (int i = 0; i < grossAmounts.size(); i++) {
    if (i < grossAmounts.size() - 1) {
        share = MoneyUtil.round(discount.multiply(grossAmounts.get(i))
                .divide(subTotal, 8, RoundingMode.HALF_UP));   // 8 dp, then round to 2
        assigned = assigned.add(share);
    } else {
        share = MoneyUtil.round(discount.subtract(assigned));   // last line = remainder
    }
    lineDiscounts.add(share);
}
```

- **Line by line**: each line's share = discount x (its gross / subtotal), rounded to 2 decimals.
- **Last line gets the remainder** - so the parts always add up to the exact discount. Without this, rounding can lose or invent 0.01.
- Why proportional: charging the whole discount on line 1 would distort line 1's GST base - GST must be computed on each line's real taxable value.

GST per line

```
taxable = round(gross - lineDiscount)
gst     = round(taxable * gstRate / 100)     // scale 8 then HALF_UP to 2
cgst    = round(gst / 2)                      // intra-state: half each
sgst    = round(gst - cgst)                   // remainder -> cgst+sgst == gst EXACTLY
```

- Example from the smoke test: taxable 190.00, GST 12% -> **22.80**, CGST **11.40**, SGST **11.40**.
- `sgst = gst - cgst` (not another division) guarantees the two halves sum to the total - no 0.01 drift.

FEFO deduction (First-Expiry-First-Out)

```
List<StockBatch> sellable = stockBatchRepository
        .findSellableByProduct(product.getId(), today);   // ORDER BY expiryDate ASC
int available = sum(batch.getQuantity());
if (available < needed)
    throw new InsufficientStockException("Not enough stock for '...' ");

for (StockBatch batch : sellable) {          // earliest expiry first
    if (remaining == 0) break;
    int take = Math.min(batch.getQuantity(), remaining);
    batch.setQuantity(batch.getQuantity() - take);
    remaining -= take;
}
```

- Check **before** deducting (fail fast), deduct in expiry order, every touched batch is a managed entity - Hibernate writes the updates at commit.
- The FEFO ordering lives in one JPQL: `StockBatchRepository.findSellableByProduct()` - `WHERE quantity > 0 AND expiryDate >= :today ORDER BY expiryDate ASC, id ASC`.
- Smoke test proof: selling Oil 1L drained batch OIL-B1 (earlier expiry) to 0 before touching OIL-B2.

Invoice number

```
// InvoiceService.nextInvoiceNumber(ownerId)
```

```
Optional<Invoice> last = invoiceRepository.findTopByOwnerIdOrderByIdDesc(ownerId);
if (last.isPresent() && parts[1].equals(String.valueOf(year)))
    next = Integer.parseInt(parts[2]) + 1;
return String.format("INV-%d-%04d", year, next);   // INV-2026-0001
```

- Format `INV-YYYY-NNNN`, sequence **restarts each calendar year**, continues from that shop's last invoice of this year.
- Simple read-and-increment, scoped to the tenant (`findTopByOwnerIdOrderByIdDesc`): perfect per shop; **not gap-free under concurrency** (I will say this before the interviewer does - see limitations).

7.2 `JwtAuthFilter.doFilterInternal()` (security/JwtAuthFilter.java:32)

```
String header = request.getHeader("Authorization");
if (header != null && header.startsWith("Bearer ")) {
    String token = header.substring(7);
    try {
        String email = jwtUtil.extractEmail(token);
        if (email != null
                && SecurityContextHolder.getContext().getAuthentication() == null
                && jwtUtil.isValid(token, email)) {
            UserDetails user = userDetailsService.loadUserByUsername(email);
            var auth = new UsernamePasswordAuthenticationToken(
                    user, null, user.getAuthorities());
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
    } catch (Exception ex) { /* continue anonymous -> 401 later */ }
}
filterChain.doFilter(request, response);
```

- **`substring(7)`** strips exactly the `Bearer ` prefix.
- The `getAuthentication() == null` guard means it never overwrites an existing login.
- Swallow-and-continue is intentional: one filter, and Spring Security's entry point produces the uniform 401 JSON.
- `OncePerRequestFilter` guarantees the filter runs **once per request** even with forwards.

7.3 Other classes worth quoting

- **`MoneyUtil.round()`** - `value.setScale(2, RoundingMode.HALF_UP)`; **all** money math in the app goes through it. One place to change rounding policy.
- **`AmountInWordsUtil.convert()`** - Indian numbering (crore/lakh/thousand) for the PDF: `21280.50 -> "Twenty One Thousand Two Hundred Eighty Rupees and Fifty Paise Only"`.
- `AuthService.register()` - three modes: anonymous callers may register only as OWNER (that starts a new shop - the old first-user bootstrap rule is gone); an OWNER token may only add STAFF, with owner_id set to the caller's shop; a STAFF token always gets 403. Email stored lowercase; password BCrypt-hashed.
- **`GlobalExceptionHandler`** - maps `ApiException -> its status`, validation errors joined into one readable string, `DuplicateKeyException -> 409`, optimistic lock -> 409 retry message, `Exception -> 500 generic` (no stack traces leaked).
- **`SecurityConfig.securityFilterChain()`** - disables CSRF, sets STATELESS sessions, declares the URL role rules, registers custom 401/403 JSON handlers, adds `JwtAuthFilter` before `UsernamePasswordAuthenticationFilter`.
- **`PdfService.generateInvoicePdf()`** - builds a lowagie `Document` in memory, Helvetica fonts, three tables (header/customer/items), totals block, amount-in-words line, footer with shop GSTIN, returns `byte[]`.
- **`ProductController.list()`** - `size = Math.min(size, 100)` - clients cannot ask for 10,000 rows in one page (cheap abuse protection).

7.4 Annotations and config you must be able to explain

- `@RestController` = `@Controller` + `@ResponseBody` (returns JSON, not a view).
- `@Transactional` - starts a DB transaction at method entry, commits at exit, **rolls back on any RuntimeException**. `readOnly = true` on queries.
- `@Valid` on a `@RequestBody` parameter - triggers Bean Validation (`@NotBlank`, `@Min`, `@DecimalMin`, `@NotEmpty` on `InvoiceRequest.items`).
- `@RestControllerAdvice` - exception handler for **all** controllers (global).
- `@Version` - Hibernate optimistic lock: it increments the column and checks it on update.
- `@Query("JPQL ...")` - custom query when derived method names are not enough; `@Param` binds arguments.
- `@Value("${app.jwt.secret}")` - injects config; the secret comes from the `JWT_SECRET` env var, never hard-coded in Java.
- `@ConfigurationProperties`-style externalization: `application.properties` uses `${DB_URL:default}` - env var wins, default keeps dev easy.
- `spring.jpa.open-in-view=false` - **no session kept open during rendering**; forces me to assemble DTOs inside the service (a real architecture decision, easy interview win).

8. How It Was Built

Development order (matches the 25 git commits)

- **Commit 1** - repo setup: PLAN.md, DECISIONS.md, .gitignore, git init.
- **Commit 2** - backend skeleton: entities, repositories, JWT auth, `SecurityConfig`.
- **Commit 3** - category/product/supplier CRUD + pagination.
- **Commit 4** - purchase entry -> stock batches; stock + alert endpoints.
- **Commit 5** - billing (FEFO + GST), invoice numbering, PDF, reports.
- **Commit 6** - `data.sql` seed (20 products, 3 suppliers, batches, invoices) + smoke test script.
- **Commit 7** - React frontend: all 10 pages wired to the API.
- **Commit 8** - 25 tests (unit + MockMvc + service).
- **Commit 9** - docs: README, INTERVIEW_NOTES, screenshots placeholder.
- **Commits 10-11** - UI redesign (custom ink/gold design system) + invoice modal bug fix.
- **Commit 12** - splash loader video and MunimJi logo branding (transparent wordmark, favicon).
- **Commits 13-15** - diagram set in UnderstsndThisProject (runtime architecture, HLD/LLD, flows), repo cleanup, interview prep document.
- **Commits 16-19** - offline demo mode: mock dataset when the backend is unreachable, dummy owner login, brand tile polish, diagram prune.
- **Commits 20-22** - login brand panel, Users and Access page wired to register, offline register support.
- **Commits 23-25** - multi-tenant backend (owner_id isolation, public shop registration, TenantIsolationTest), Register page, anonymous-register fix.
- Approach: **walking skeleton first** - one thin slice through every layer (auth), then feature-by-feature depth (billing core after basic CRUD).

Challenges faced and how I solved them (from real code/commits)

- **Rounding drift in GST** - naive per-line splits could lose 0.01. Solved: divide at scale 8 then round, and give the **remainder to the last line** so parts sum exactly.
- **Two batches per product** - a single `quantity` column could not express FEFO. Solved: `stock_batch` table + JPQL ordered by expiry; smoke test drains B1 before B2.
- **Rollback proof** - if stock deducts but the invoice fails, the shop loses stock. Solved: everything inside one `@Transactional` method; smoke script asserts stock is unchanged after a forced failure.
- **Test DB pollution** - MockMvc tests shared one H2 database and leaked the logged-in user across tests (SecurityContext is thread-local). Solved: `@Transactional` rollback on every test class + `SecurityContextHolder.clearContext()` in `@AfterEach`.
- **Flaky JSON decimal asserts** - asserting `135.72` through json-smart doubles was unstable. Solved: assert strings and integers (stock quantities, invoice numbers) instead of raw decimals.
- **Smoke script not re-runnable** - it created "Filter Oil 1L" each run, so seed-count checks failed on a dirty database. Solved: documented that it needs a fresh DB (delete `backend/data`, restart).
- **Git ref corruption (real incident)** - `.git/refs/heads/master` was accidentally zero-filled; git reported "your current branch appears to be broken". Solved: repaired the ref by writing the known commit SHA back into the file, verified with `git log`. (Uncertain detail: the exact tool/command that corrupted it - I could not reproduce it.)
- **Invoice modal clipped (real bug after redesign)** - the page wrapper used a CSS animation with `fill-mode: both` ending at `transform: translateY(0)`. That invisible transform made the wrapper the containing block for `position: fixed`, so the modal was trapped inside page content. Solved: end keyframes at

`transform: none` **and** render `Modal` through `createPortal(..., document.body)` so no ancestor can ever clip it.

- **jjwt 0.11.5 API trap** - must build the parser with `.build()` before `.parseClaimsJws(token)` or it throws. Noted in DECISIONS.md so I answer it from memory.
- **Multi-tenancy retrofit** - adding `owner_id` after the fact meant touching every repository query and service. Solved: `AppUserDetails` carries the tenant id in the principal, `CurrentTenant.id()` feeds each scoped query, cross-tenant reads answer 404. Locked in with `TenantIsolationTest` (6 tests).

> **Interview Tip:** Tell bug stories in three beats: symptom -> root cause -> fix. The CSS-transform modal bug is a great closer: it shows you debug with reasoning, not guessing.

Git history insights

- 25 commits, **small and feature-scoped** with conventional prefixes (`feat:`, `fix:`, `test:`, `docs:`, `chore:`).
- Verification after every step is written into PLAN.md: `mvn clean test`, `npm run build`, smoke script - proof lives in PROGRESS.md.
- No force-pushes/amends; each stage of the project is recoverable - I can show `git show <sha>` live if asked.

9. Security, Performance and Error Handling

Security checklist (what is actually in the code)

- **Passwords**: BCrypt with salt (`BCryptPasswordEncoder` bean). Never stored/returned in plain text; login errors are generic ("Invalid email or password").
- **Tokens**: JWT HS256, 24 h expiry (`app.jwt.expiration-ms=86400000`), subject = email, claims = role + name. Secret from `JWT_SECRET` env var (`@Value`), never in Java source.
- **Stateless**: `SessionCreationPolicy.STATELESS` - no server session, any number of app instances can serve requests.
- **CSRF disabled** - deliberate: the token sits in `Authorization` header from JavaScript, not a cookie; CSRF attacks target cookie-authenticated apps.
- **CORS whitelist**: only `http://localhost:5173`, methods GET/POST/PUT/DELETE/OPTIONS, headers Authorization + Content-Type (`SecurityConfig.corsConfigurationSource()`).
- **Authorization**: deny-by-default - explicit allow-list for STAFF reads + invoice create, reports/OWNER for everything else, `anyRequest().hasRole("OWNER")` as the safety net.
- **Tenant isolation** - every service reads the caller's shop id from `CurrentTenant.id()` and passes `owner_id` into each query; a cross-tenant id returns the same 404 as a missing row, so enumeration leaks nothing (`TenantIsolationTest`). The frontend never chooses the shop - it is derived from the JWT.
- **Input validation**: Jakarta validation on DTOs (`@NotBlank`, `@DecimalMin`, `@NotEmpty`, sizes); business checks in services (negative discount -> 400, discount > subtotal -> 400).
- **Error responses**: single shape `{status, message}`; generic 500 message - **no stack traces** in responses.
- Frontend stores JWT in **localStorage** - simple but XSS-visible; honest limitation with mitigation notes (CSP, httpOnly+refresh tokens as the upgrade path).

Performance considerations (what I did / did not do)

- **Pagination** on products (`GET /api/products?page&size&search`, size capped at 100) and invoices (`GET /api/invoices?page&size`), wrapped in `PagedResponse {content, page, size, totalElements, totalPages}`.
- **N+1 prevention**: `JOIN FETCH` for invoice-with-items, purchase-with-details, product-with-batches; `DISTINCT` on collection joins; `open-in-view=false` so lazy loading happens where I plan it.
- **LAZY everywhere** by default (`@ManyToOne(fetch = LAZY)`), so a list query does not drag categories/batches along.
- **DB portability over DB power for reports**: Java loops instead of `GROUP BY` - fine at shop scale (thousands of invoices), and I state the SQL-aggregation upgrade path.
- **Frontend**: Recharts split into its own bundle chunk (`manualChunks` in `vite.config.js`); no caching layer, no `@Cacheable` - **honest: not implemented**.
- Money is `BigDecimal` end-to-end - never float (accuracy beats micro-speed).

Error handling architecture

- One base: `ApiException extends RuntimeException` carrying `HttpStatus`; subclasses: `BadRequest 400`, `Unauthorized 401`, `Forbidden 403`, `ResourceNotFound 404`, `Conflict 409`, `InsufficientStock 400`.
- `GlobalExceptionHandler` (@RestControllerAdvice) maps them all to `{status, message}`; validation errors are **joined into one readable line** (`name is required; gstRate must be 0 or more`).

- Special cases: `DuplicateKeyException -> 409`, `ObjectOptimisticLockingFailureException -> 409` "Stock was updated by someone else. Please retry.", bad JSON -> 400, fallback `Exception -> 500` generic.
- Security layer answers **before** controllers: 401 "Please login first", 403 "You don't have permission" - written by custom entry point / access-denied handler in `SecurityConfig`.
- Frontend: `errorMessage(err)` extracts `data.message` (or "Cannot reach the server") and pages render it in an alert box - users never see raw axios errors.

> **Note:** Logging: Spring Boot default auto-configuration only (startup logs, Hikari). No custom log files or log levels configured in the repo - say "default logging" rather than claiming a logging framework.

10. Testing, Deployment and DevOps

Automated tests: 34 in 9 classes (brief asked 15)

- **MoneyUtilTest (2)** - rounding HALF_UP to 2 dp, zero scale correctness.
- **AmountInWordsUtilTest (2)** - amount-in-words output for the PDF.
- **AuthControllerTest (7)** - login success, wrong password 401, and the five register rules (anonymous owner 201, anonymous staff 403, owner token cannot mint an owner, staff token 403, owner adds staff 201).
- **SecurityRulesTest (5)** - STAFF 403 on writes, STAFF allowed reads/invoice create, reports OWNER-only, unauthenticated 401.
- **TenantIsolationTest (6)** - two owners cannot read each other's invoices or bill against each other's products, category names unique per shop, staff scoped to their owner's shop, reports scoped to the shop.
- **ProductApiTest (4)** - create/list/search/pagination/validation 400.
- **InvoiceBillingTest (4)** - GST/discount math, FEFO across two batches, insufficient stock rollback, invoice number.
- **PdfServiceTest (1)** - PDF bytes start with `%PDF` and contain content.
- **StockApiTest (3)** - low stock, expiring, expired alerts.
- Technologies: `@SpringBootTest` + **MockMvc** (full context, no real server), `@DataJpaTest`-style assertions inside service tests, `spring-security-test` where needed.
- Test config: `src/test/resources/application-test.properties` - in-memory H2 (`jdbc:h2:mem:munimji-test;MODE=MySQL;DB_CLOSE_DELAY=-1`), `ddl-auto=create-drop`, seed still runs. Result: `mvn clean test` -> **Tests run: 34, Failures: 0, Errors: 0**.
- Every test class is `@Transactional` (rolls back after each test) and clears `SecurityContextHolder` in `@AfterEach`.

End-to-end smoke test (the definition-of-done script)

- `scripts/smoke-test.ps1` - PowerShell + `curl.exe`, **62 checks**, exit code 0 only if all pass.
- Covers: login -> seed counts -> CRUD + purchase -> billing FEFO & GST (190 x 12% = 22.80 split 11.40/11.40) -> failure cases (insufficient stock, expired batch never sold) -> register rules (anonymous owner, owner-adds-staff, staff refused) -> STAFF role rules -> PDF magic bytes -> pagination -> reports -> invoice detail.
- Last run on a fresh database: **ALL 62 CHECKS PASSED** (proof in PROGRESS.md).
- It is **not idempotent** (it creates data), so run it against a fresh DB - I documented this in README.

How to run / environment configs

```
# Backend (port 8080)
mvn spring-boot:run            # dev
mvn clean package              # -> target/munimji-backend-1.0.0.jar
java -jar target/munimji-backend-1.0.0.jar
mvn clean test                 # 34 tests

# Frontend (port 5173)
npm install
npm run dev                    # Vite dev server
npm run build                  # production bundle in frontend/dist

# MySQL instead of H2 (env vars - no code change)
DB_DRIVER / DB_URL / DB_USERNAME / DB_PASSWORD

# Other env knobs
JWT_SECRET        # JWT signing secret (change before production!)
SEED_MODE=never   # skip sample data
```

```
DB_DDL=validate   # stricter schema check
SHOP_NAME / SHOP_GSTIN / SHOP_ADDRESS / SHOP_PHONE   # printed on PDF
```

- Seed accounts (data.sql, BCrypt hashes): `owner@munimji.local / owner123` (OWNER), `staff@munimji.local / staff123` (STAFF).
- Seed data: 6 categories, 3 suppliers, **20 products**, 20 stock batches (varied expiry), 3 purchases, 2 sample invoices.

Deployment and CI/CD - honest status

- **Deployment in repo**: runnable jar + static frontend build; README quick start. That is what exists.
- **Docker / docker-compose**: Not found in repo.
- **CI/CD (.github/workflows)**: Not found in repo - my answer: "I would add a GitHub Actions workflow: `mvn clean test` + `npm run build` + smoke test on every push."
- Production shape I would describe: jar behind Nginx, React `dist` served as static files, MySQL with env secrets, HTTPS, managed backups.

> **Common Mistake:** Never claim Kubernetes/Docker/Jenkins if the repo has none. "Not yet - here is exactly what I would add and why" is the correct, credible answer.

11. Limitations, Improvements and Future Scope

Honest weaknesses in the current code

- **Invoice numbers are read-and-increment** (`findTopByOwnerIdOrderByIdDesc`) - two bills at the exact same moment could collide; fine for one counter, not for many. Fix: DB sequence per year or `SELECT ... FOR UPDATE`.
- **No refresh token / logout server-side** - access token lives 24 h; stolen token works until expiry; no revocation list. Fix: short-lived access + refresh token rotation.
- **JWT in localStorage** - readable by any XSS. Fix: httpOnly Secure cookies + CSRF protection, or strict CSP.
- **GST is intra-state only** (CGST/SGST) - IGST / inter-state place-of-supply rules are not modelled.
- **Reports loop in Java** over fetched invoices - O(n) in memory; at millions of rows this must become SQL `GROUP BY` or materialized views.
- **Invoice item links only to the FIRST batch used** when one line spans two batches (deduction still happens correctly on both; the reference is cosmetic but can mislead audits).
- **No explicit DB indexes** (schema from Hibernate); no read replicas, no cache.
- **No payment/credit module** - bills are immediate sale records; no dues (baki khata), no returns/refunds, no GST e-invoice/IRN integration.
- **No audit trail** - no created_by/updated_by on invoices/purchases.
- **No CI/CD or container setup**; tests do not include frontend (no React tests).

How I would scale to 1 lakh / 1 million users

- **Database first**: MySQL/Postgres cluster, add the indexes named in section 6, move report aggregation to SQL, connection pool tuning (Hikari), read replicas for reports.
- **Stateless API** - JWT already makes every node identical: add N app instances behind a load balancer; sessions are not a problem because there are none.
- **Cache the hot paths**: product search + dashboard summary behind Redis or a simple Caffeine cache; invalidate on write. (Not in the stack - adding it is a decision, not a default.)
- **Async heavy work**: PDF generation and monthly GST reports onto a queue/worker so HTTP threads stay fast.
- **Frontend**: CDN-hosted static build, code-split by route (already chunked), paginate/virtualize big tables.
- **Rate limiting + WAF** in front; horizontal pod autoscaling; blue-green deploys.
- **Shard by owner_id** when one database struggles - tenancy is already row-level (owner_id on every shop-scoped table), so moving a shop's rows to its own shard is mechanical, not a redesign.

What I would change if I rebuilt it

- Invoice numbering with a **`invoice_sequence` table row updated with `FOR UPDATE`** (gap-free, still simple).
- **Refresh tokens + revocation**; move auth out of localStorage.
- Reports as **SQL aggregates with EXPLAIN-verified indexes**; keep Java loops only for tiny tables.
- **IGST + place of supply**, HSN-wise summary ready for GSTR filing.
- Add **payments, returns, and audit columns** - the real shopkeeper's next asks.
- **Dockerfile + GitHub Actions** from day one; Testcontainers for MySQL integration tests.
- Multi-tenant `owner_id` from commit one - I added it mid-project and it worked, but touching every repository query is exactly the pain I would design out up front.

> **Remember:** A limitation you raise yourself is a strength. Always pair it with the fix: "today's code does X; I would change it to Y because Z."

12. Interview Q&A Bank

140 questions with sample answers, follow-ups and Remember tips. Answers are written in **your voice** - the fresher who built this exact repo. Rehearse out loud; do not memorise word by word.

> **Interview Tip:** In the real interview, say the answer in 3-4 sentences and stop. The follow-up list shows what they ask NEXT - be ready for it, do not dump it all in your first reply.

a) Project overview and motivation

Q1. Tell me about this project.

Answer: **MunimJi** is an inventory and GST billing app I built solo for small kirana-style shops. Backend is **Spring Boot 3.3 + Java 17** with layered controllers/services/repositories, **JWT auth** with OWNER and STAFF roles, and an **H2/MySQL** database, every query scoped by owner_id. Frontend is **React 18 + Tailwind** with a POS billing screen, product and supplier management, stock alerts and reports. The core is billing: GST math, proportional discount split and **FEFO stock deduction all in one transaction**, covered by 34 tests and a 62-check smoke script.

Follow-up: What is FEFO? / How long did it take you?

Remember: Open with problem -> solution -> core highlight. End at FEFO so they ask about it - that is your best story.

Q2. Why did you build this project?

Answer: I wanted one project that touches the **full stack of real business problems**: money precision, stock with expiry dates, GST rules, roles and security. Billing gave me a hard domain (rounding, tax split, concurrency), and building everything myself - from schema to PDF - proved I can ship a complete feature, not just CRUD tutorials.

Follow-up: Why not a to-do app or e-commerce?

Remember: Small-shop framing shows empathy with users - managers like domain-motivated engineers.

Q3. What was your role and team size?

Answer: It was **solo** - I designed the schema, wrote the backend, frontend, tests, smoke script and docs. That meant I owned every decision: no one to blame and no one to unblock me, so I kept a written plan (PLAN.md) and a decisions log (DECISIONS.md) to stay disciplined.

Follow-up: How did you manage yourself without a team? / Would you work differently in a team?

Remember: Turn solo into a strength: full ownership + written discipline. Do not pretend it was a team.

Q4. Who is the target user?

Answer: A **small shop owner and their billing staff**. The owner manages products, suppliers, purchases and reports; the clerk only searches products, checks stock and prints bills - which is exactly what the **OWNER vs STAFF** role split enforces in `SecurityConfig`.

Follow-up: Why only two roles? / How would you add a third?

Remember: Roles map to real people in the shop, not to abstract admin/user.

Q5. What is unique about this project compared to others?

Answer: Most portfolio projects stop at CRUD. Mine has a **real domain core**: batch-wise stock with expiry, FEFO deduction, GST with exact rounding guarantees, invoice numbering, PDF output, and **failure semantics** - insufficient stock rolls the whole bill back. Everything is proven by an end-to-end smoke script, not just unit tests.

Follow-up: What is the hardest part of that core?

Remember: Unique = correctness under failure, not fancy tech.

Q6. Is this a real deployed product?

Answer: It runs fully on a machine - `mvn spring-boot:run` plus `npm run dev`, or a packaged jar plus a static frontend build. It is **not deployed to the cloud** yet; there is no Docker or CI/CD in the repo. For production I would containerize it, add GitHub Actions, and put it behind Nginx with MySQL and HTTPS.

Follow-up: What would deployment look like exactly?

Remember: Be exact about what exists - deployed vs deployable.

Q7. How long did the project take?

Answer: About **two to three weeks** of focused daily work, built feature by feature in 25 commits: skeleton and auth first, then CRUD, purchases and stock, billing and PDF, seed and smoke tests, frontend, tests, docs, a UI redesign, an offline demo mode, and finally the multi-tenant refactor. PLAN.md tracks that order and each step has verification proof in PROGRESS.md.

Follow-up: Which part took the longest? / What would you do faster next time?

Remember: Cite the artifacts (PLAN.md, 25 commits) - dates are fuzzy, evidence is not.

Q8. What did you learn from this project?

Answer: Three things. **Money is unforgiving** - I learned why rounding policy must live in one utility. **Transactions are the safety net** - FEFO without rollback would corrupt stock. And **wrote-down decisions speed everything up** - my DECISIONS.md stopped me re-deciding the same trade-offs weekly.

Follow-up: Any technical lesson you will never forget?

Remember: Pair each lesson with a concrete moment from the code.

Q9. What would you add next?

Answer: Short term: refresh tokens, gap-free invoice numbering, and SQL-based reports with indexes. Medium term: payments and credit (baki khata), returns, IGST for inter-state sales, and an audit trail. The multi-tenant refactor I have since built - one deployment now serves many isolated shops - while Docker and CI/CD are still missing.

Follow-up: Which would you do first and why?

Remember: Order your roadmap by risk (security) before features.

Q10. How is this useful for a real business?

Answer: It **saves money directly**: FEFO sells near-expiry stock first, expiry alerts reduce write-offs, low-stock alerts prevent lost sales, and GST invoices plus a monthly GST summary cut accountant time. For a small shop that is measurable value, not demo value.

Follow-up: How would you measure the value?

Remember: Tie features to rupees saved - that is the managerial framing.

b) Why questions (stack, DB, architecture, approaches)

Q11. Why Spring Boot and not plain Servlet/JSP?

Answer: Spring Boot gives embedded Tomcat, auto-configuration, externalized config and Spring's DI, security and data support in one dependency. I still control the boring parts explicitly - `application.properties`, beans, filter chain - so it is magic-free. Plain servlet code would mean hand-writing JSON parsing, security filters and connection pooling.

Follow-up: What does Spring Boot auto-configure that you rely on?

Remember: Name 3 auto-configs: DataSource (Hikari), Jackson, embedded Tomcat.

Q12. Why Maven?

Answer: Maven is the standard in the Spring ecosystem: fixed lifecycle, huge repository, and every recruiter knows `mvn clean test`. For this project I needed exactly that one-command verification. Gradle is faster for large multi-module builds, but a fresher portfolio gains nothing from that trade-off.

Follow-up: What does `mvn clean package` do?

Remember: Clean = remove target, package = build jar; test phase runs the 34 tests.

Q13. Why H2 as the default database?

Answer: So anyone can **clone and run with zero setup** - H2 is an embedded file database under `backend/data/`. The same code switches to **MySQL with 4 environment variables** (no code change), because the datasource URL comes from `${DB_URL:default}` in `application.properties`. H2 runs in MySQL compatibility mode (`MODE=MySQL`) so the SQL stays portable.

Follow-up: Is H2 production ready? / What may differ on MySQL?

Remember: Default = developer experience; production path = MySQL. You have both.

Q14. Why JPA/Hibernate instead of writing SQL?

Answer: Entities map the schema, JpaRepository gives CRUD and derived queries for free, and JPQL keeps queries model-focused. Where SQL precision mattered I wrote custom `@Query` methods - FEFO ordering, JOIN FETCH, date ranges. So: ORM for the 80%, hand-written JPQL for the 20% that decides correctness.

Follow-up: What is the cost of using Hibernate?

Remember: Answer with the 80/20 balance - shows you are not an ORM evangelist.

Q15. Why JWT instead of session cookies?

Answer: JWT is **stateless**: the server stores nothing, so I can scale by adding identical instances behind a load balancer. Each request carries `Authorization: Bearer`, verified by `JwtAuthFilter`. The trade-off - no instant server-side revocation - I state as a known limitation with refresh tokens as the fix.

Follow-up: What is the downside of JWT you must handle?

Remember: Name the downside first (no revocation), then the mitigation.

Q16. Why store the token in localStorage?

Answer: Simplicity: the Axios interceptor reads it for every request and a 401 clears it and redirects to login - one code path. The cost is XSS visibility, which I acknowledge; the production upgrade is an httpOnly Secure refresh cookie plus strict CSP. For a portfolio app served on localhost, the simpler path was defensible.

Follow-up: Is that secure? What attack is it exposed to?

Remember: Never defend localStorage as secure - defend it as a conscious trade-off.

Q17. Why React and not Angular or Vue?

Answer: React's component model fit my UI: shared Modal, Sidebar, Loader, StatCard; pages are just functions of state. The ecosystem (Recharts, Router, Axios) is the largest, and job demand is highest. Angular would be heavier for a solo app; Vue is equally fine but React maximised my learning value.

Follow-up: What problem does React's model solve here?

Remember: Component reuse + hiring market = honest answer.

Q18. Why no Redux? Don't you need global state?

Answer: I need exactly two pieces of global state: **token and user**. That is one `AuthContext` with `useState` in `AuthContext.jsx`. Everything else is page-local state (cart, filters, modals). Adding Redux would be a dependency and indirection for two variables - if the app grew collaborative features (live cart, notifications), I would reconsider.

Follow-up: When would you add Redux?

Remember: Draw the line: many cross-cutting states -> Redux; two -> Context.

Q19. Why Tailwind instead of a component library?

Answer: Tailwind let me build a **unique design system** - my own ink/brand/gold palette, `.card`, `.btn`, `.input` classes in `index.css` - without fighting a library's opinions. Utility classes also keep styles next to the markup, so there is no CSS hunting. Bootstrap or MUI would have made every screen look generic.

Follow-up: Isn't Tailwind verbose? / How do you keep it DRY?

Remember: Mention the `@layer components` classes - that is your DRY answer.

Q20. Why BigDecimal for money and not double?

Answer: Doubles are binary floats - 0.1 + 0.2 does not equal 0.3, and GST rounding errors compound across lines. `BigDecimal` with `setScale(2, HALF_UP)` through **one utility (`MoneyUtil.round`)** gives predictable decimal arithmetic, and DB columns are declared `precision 19, scale 2` to match.

Follow-up: Why scale 8 in some divisions?

Remember: Divide at 8 dp to avoid premature rounding, then round to 2 - that is the precise answer.

Q21. Why OpenPDF for invoices?

Answer: It is free (LGPL), pure Java, and gives me full layout control - I place the shop header, GSTIN, item table, totals and an amount-in-words line. iText's licensing would be a problem for commercial use, and external tools like wkhtmltopdf add a server dependency. OpenPDF keeps the build self-contained.

Follow-up: How do you test PDF output?

Remember: Answer: assert `%PDF` magic bytes and content presence (PdfServiceTest).

Q22. Why monolith and not microservices?

Answer: This is a single shop's app with one database and heavy transactional coupling between billing and stock - splitting them would mean distributed transactions for no benefit. A modular monolith with clean layers gets 90% of the value at 10% of the ops cost. I would extract services only when scale or team boundaries demand it.

Follow-up: At what point would you split it?

Remember: Say the trigger: separate scaling domains or independent teams - not fashion.

c) How questions (flows and mechanisms)

Q23. How does login work end to end?

Answer: `POST /api/auth/login` -> `AuthController` -> `AuthService.login()` calls `AuthenticationManager.authenticate()`, which compares the password against the **BCrypt hash** via `UserDetailsService`. On success `JwtUtil.generateToken()` builds an HS256 token with `sub = email` and claims `role`/`name`, 24 h expiry. React stores it in localStorage; the Axios interceptor attaches `Authorization: Bearer` to every call.

Follow-up: What happens on wrong credentials? / Where is the hash stored?

Remember: Mention the generic error: "Invalid email or password" - no user enumeration.

Q24. How does the server know who is calling on each request?

Answer: `JwtAuthFilter` (registered before `UsernamePasswordAuthenticationFilter`) reads the Authorization header, extracts and validates the token with `JwtUtil.isValid()` - signature, subject match, not expired - loads the user, and sets the authentication into `SecurityContextHolder`. Controllers then run as that principal, and URL rules decide 401/403.

Follow-up: What if the token is missing or expired?

Remember: Bad token -> filter continues anonymously -> rules answer 401 uniformly.

Q25. How does role-based authorization work?

Answer: In `SecurityConfig.securityFilterChain()`: `permitAll` for login/register; GET products/categories/stock and GET+POST invoices allow **OWNER or STAFF**; `/api/reports/**` and `anyRequest()` require **OWNER**. Spring Security maps roles to `ROLE_OWNER`/`ROLE_STAFF` from the JWT-backed UserDetails. Denied -> 403 JSON from my custom access-denied handler.

Follow-up: How do you test this? / How would you add a MANAGER role?

Remember: Deny-by-default is the key phrase: only explicit rules are opened.

Q26. How does creating an invoice work?

Answer: One `@Transactional InvoiceService.create()`: validate discount, load products and compute line gross, split the discount proportionally (remainder to last line), compute taxable/GST/CGST/SGST per line, check FEFO availability then deduct from earliest-expiry batches, generate `INV-2026-0001`, snapshot item details, cascade-save invoice + items. Any exception rolls everything back.

Follow-up: What if stock runs out on the second line after the first was deducted?

Remember: Rollback undoes the first deduction too - say it explicitly.

Q27. How exactly does FEFO deduction work?

Answer: The JPQL in `StockBatchRepository.findSellableByProduct()` returns batches with `quantity > 0` and `expiryDate >= today`, **ordered by expiry date ascending**. I sum the quantities for an availability check first; if enough, I walk the list taking `min(batch.quantity, remaining)` until the need is met. Earliest-expiring stock always sells first.

Follow-up: What about expired batches?

Remember: Expired rows are filtered out entirely - expired stock can never be sold (smoke-tested).

Q28. How is GST calculated?

Answer: Per line: `taxable = gross - discountShare`, `gst = taxable * gstRate / 100` (scale 8, HALF_UP, rounded to 2). Then `cgst = gst / 2` and `sgst = gst - cgst`, so the halves sum exactly. Example: 190.00 at 12% -> 22.80 GST -> 11.40 + 11.40. Invoice totals are the sum of line values.

Follow-up: Why not IGST? / What if gstRate is 0?

Remember: The `gst - cgst` trick is the memorable detail - expect a follow-up.

Q29. How does the discount split work, and why is it split at all?

Answer: Each line's share = `discount * (lineGross / subtotal)` rounded to 2 decimals, except the **last line which takes the remainder** so shares add up exactly. Splitting matters because GST is computed on each line's taxable value after discount - putting the whole discount on one line would misstate that line's tax.

Follow-up: Give me a rounding example that would break naive code.

Remember: Three-way split of 10.00 over lines can leave 0.01 unassigned - remainder fixes it.

Q30. How are invoice numbers generated?

Answer: `nextInvoiceNumber()` reads the newest invoice (`findTopByOwnerIdOrderByIdDesc`), parses `INV-YYYY-NNNN`, and if its year matches the current year increments the counter, else starts at 1. Format is `String.format("INV-%d-%04d", year, next)` - so January resets the sequence.

Follow-up: Is this safe with two cashiers at once?

Remember: Answer honestly: single-counter safe, not gap-free under concurrency - upgrade path is a sequence table.

Q31. How does purchasing stock work?

Answer: `POST /api/purchases`: `PurchaseService.create()` verifies the supplier, then for each line saves a `PurchaseItem` **and creates a fresh `StockBatch`** with batch number, expiry, quantity and cost price; totals are

`sum(cost * qty)`. All inside one transaction with cascade save. That batch immediately becomes visible to FEFO.

Follow-up: What if the supplier does not exist?

Remember: 404 `ResourceNotFoundException` - services throw, advice maps.

Q32. How do stock alerts work?

Answer: `StockService` exposes four reads: `viewStock()` (totals per product from batches), `lowStock()` (total quantity below the product's `reorderLevel`), `expiring(days)` (`findExpiring` with `expiryDate` between today and today+days), and `expired()` (`expiryDate < today`). The frontend shows them as tabs with counts on the Stock page.

Follow-up: Where does the default 45 days come from?

Remember: It is a request parameter with a default - the UI sends it.

Q33. How does the frontend talk to the backend safely?

Answer: One Axios instance in `src/api/axios.js` with base URL `http://localhost:8080/api`. A **request interceptor** adds `Bearer <token>` from localStorage (skipped when the call sets config.anonymous - the register call does, so a lingering owner token cannot turn sign-up into a 403); a **response interceptor** catches 401, clears the stored token and redirects to `/login`. Pages never touch headers themselves, and errors go through `errorMessage()` so users see friendly text.

Follow-up: What about 403 - does the frontend handle it?

Remember: 403 shows the server's message; UI also hides owner-only buttons via `isOwner` - cosmetic only.

Q34. How is the PDF invoice generated and downloaded?

Answer: GET `/api/invoices/{id}/pdf` loads the invoice with items, `PdfService.generateInvoicePdf()` builds a lowagie `Document` into a `ByteArrayOutputStream` - shop header, GSTIN, customer block, item table, totals, amount in words - and returns `byte[]` as `application/pdf`. The frontend does a **blob fetch with the auth header** (a plain link cannot send JWT), then triggers the download.

Follow-up: How do you know the PDF is correct?

Remember: Smoke checks: `%PDF` prefix, `%%EOF` suffix, content; PdfServiceTest asserts bytes.

Q35. How does pagination work?

Answer: Products: `GET /api/products?search=&page=0&size=10` - the controller caps `size` at 100, the repository runs a JPQL `search` with `JOIN FETCH category` and `DISTINCT`, returning Spring `Page`. Invoices use `findAllSummaries` as a constructor projection so the list view loads only summary fields. Both wrap into `PagedResponse {content, page, size, totalElements, totalPages}`.

Follow-up: Why a projection for invoices?

Remember: List view does not need line items - projection avoids loading the collection.

Q36. How do you debug a failing endpoint?

Answer: Four steps: reproduce with curl/POSTman to see the raw status and `{status, message}`; check the backend log for the stack trace; trace the layer - controller mapping, service rule, repository query - with a breakpoint or temporary log; fix and re-run the matching test plus `mvn clean test`. For frontend issues I use browser dev tools: network tab first, console second.

Follow-up: What if it only fails in the UI?

Remember: Network tab shows the real status code - catches 90% of 'frontend bugs'.

d) Code-level questions (specific classes, methods, annotations)

Q37. Explain `InvoiceService.create()` line by line.

Answer: It is `@Transactional`. Step 1: validate discount (non-negative, <= subtotal). Step 2: load each product, compute `gross = sellingPrice * qty`, accumulate subtotal. Step 3: split discount proportionally with remainder on the last line. Step 4: per line compute taxable, GST, CGST, SGST. Step 5: FEFO availability check then deduct batches. Step 6: build `Invoice` with number and totals, build `InvoiceItem`s with snapshots, cascade save, return `InvoiceResponse`.

Follow-up: Where can it throw, and what happens then?

Remember: Throw points: BadRequest, ResourceNotFound, InsufficientStock - all trigger rollback.

Q38. What does the discount-split loop do exactly? Why scale 8?

Answer: For every line except the last: `share = round(discount * gross / subtotal)` with `divide(subTotal, 8, HALF_UP)` - dividing at **8 decimal places** first avoids rounding each share too early; then `round` brings it to 2 dp. The last line gets `discount - assigned`, so the sum of shares is mathematically equal to the discount.

Follow-up: What if there is only one line?

Remember: Single line = it IS the last line -> gets the whole remainder; loop still correct.

Q39. Explain `JwtAuthFilter` - why `OncePerRequestFilter`?

Answer: It reads `Authorization`, strips `Bearer `, validates via `JwtUtil`, and populates `SecurityContextHolder` if empty. `OncePerRequestFilter` guarantees execution exactly once per request (no double execution across forwards/commits). Exceptions are swallowed on purpose - an invalid token simply means the request proceeds anonymous and gets a 401 from the security rules.

Follow-up: Why not throw 401 directly from the filter?

Remember: Centralizing 401/403 in SecurityConfig keeps error bodies consistent.

Q40. How does `JwtUtil.generateToken` work?

Answer: Claims map carries `role` and `name`; `setSubject(user.getEmail())` is the identity; `setIssuedAt`/`setExpiration` give a 24-hour life from `app.jwt.expiration-ms`; `signWith(getSigningKey(), HS256)` signs with `Keys.hmacShaKeyFor(secret)` built from the configured secret. Parsing on the other side builds the parser with `.build()` before `parseClaimsJws(token)` - a jjwt 0.11.5 requirement.

Follow-up: HS256 vs RS256 - which and why?

Remember: HS256 shared secret is fine for one service validating its own tokens; RS256 when multiple services verify.

Q41. What does `GlobalExceptionHandler` handle?

Answer: `ApiException` -> its own status; `MethodArgumentNotValidException` -> 400 with field errors joined (`name is required; gstRate must be 0 or more`); `DuplicateKeyException` -> 409; `ObjectOptimisticLockingFailureException` -> 409 "Stock was updated by someone else. Please retry."; `HttpMessageNotReadableException` -> 400 bad JSON; and a catch-all -> 500 with a **generic** message so internals never leak.

Follow-up: Why join validation messages instead of a list?

Remember: One message string matches one UI alert box - simple and user-friendly.

Q42. What is the difference between `ApiException` and Spring's `ErrorResponseException` here?

Answer: `ApiException` is my own base carrying an `HttpStatus`; six subclasses (`BadRequest 400`, `Unauthorized 401`, `Forbidden 403`, `ResourceNotFound 404`, `Conflict 409`, `InsufficientStock 400`) let services throw business-meaningful errors without knowing HTTP plumbing. The advice converts them into the uniform `{status, message}` body. It is intentionally simpler than Spring's exception interfaces.

Follow-up: Where is it thrown? Give an example.

Remember: `InsufficientStockException` in FEFO check -> 400 with the available/requested quantities.

Q43. Why are entities never returned from controllers?

Answer: Controllers only move **DTOs**: `ProductRequest` in, `ProductResponse` out. That prevents JSON cycles (invoice -> items -> invoice), hides DB internals, lets me snapshot/derive fields (like per-item CGST/SGST computed in `toResponse`), and lets the schema evolve without breaking API clients. It is the DTO pattern applied at every boundary.

Follow-up: Isn't writing DTOs boilerplate-heavy?

Remember: Trade-off acknowledged: verbosity for API stability - no Lombok, so constructors are explicit.

Q44. What does `@Transactional(readOnly = true)` actually buy you?

Answer: It opens a read-only transaction: Hibernate can skip dirty checking, some JDBC drivers hint the database, and lazy loads inside the method are guaranteed to work within the transaction. On H2 the gain is small; on MySQL/Postgres with replication it lets the driver route to a replica. Semantically it also signals: this method must not write.

Follow-up: What happens if you write inside a readOnly method?

Remember: Hibernate will throw at flush - a useful self-check.

Q45. Explain the `@Version` field on StockBatch.

Answer: Hibernate treats it as an **optimistic lock** column: on UPDATE it appends `WHERE version = ?`. If another transaction already changed the batch, zero rows update -> `ObjectOptimisticLockingFailureException` -> my advice returns 409 "Stock was updated by someone else. Please retry." It prevents two counter clerks from both selling the last unit silently.

Follow-up: Optimistic vs pessimistic locking?

Remember: Optimistic = retry-friendly, no row locks held; pessimistic = `SELECT FOR UPDATE` when conflicts are frequent.

Q46. Why is `open-in-view=false`?

Answer: Spring Boot's default keeps the Hibernate session open for the whole HTTP request, which lets lazy loads happen anywhere - including accidentally in a view layer - hiding N+1 problems until production. Turning it off forces every needed association to load inside the service (via JOIN FETCH), keeping the data access plan explicit and the session short.

Follow-up: Does it break anything?

Remember: Only breaks lazy access outside a transaction - my services map to DTOs before returning.

Q47. Why is `cascade = ALL, orphanRemoval = true` on Invoice.items?

Answer: Saving the `Invoice` automatically persists its `InvoiceItem`s - I add items to the in-memory list and call `save(invoice)` once. `orphanRemoval` means removing an item from the list deletes its row on flush. For billing this is a single aggregate: an item without its invoice is meaningless.

Follow-up: Any danger?

Remember: Cascade on child collections is fine; cascading REMOVE upward (item -> invoice) would be wrong - I did not.

Q48. How does `MoneyUtil` keep money correct?

Answer: Every arithmetic result passes through `MoneyUtil.round()` -> `setScale(2, HALF_UP)`, with constants `TWO` and `HUNDRED` so divisions share one policy. `zero()` returns a properly scaled zero so sums keep the same scale. Rounding policy in one place means I can change it once - that single choke point is the design.

Follow-up: HALF_UP vs HALF_EVEN - which is better?

Remember: HALF_EVEN (banker's) is more statistically fair; HALF_UP matches Indian retail expectation - I chose convention.

Q49. What does `ProductController.list()` do about huge page sizes?

Answer: `size = Math.min(size, 100)` - a client asking for `size=100000` silently gets 100. It is a one-line guard against accidental or hostile full-table loads. Combined with `search` and `page` params it keeps the products endpoint predictable without rate limiting.

Follow-up: Should it reject instead of clamp?

Remember: Either is defensible; clamping keeps naive clients working - state the trade-off.

Q50. How does `ProductService.delete()` protect data?

Answer: Before deleting it checks `purchaseItemRepository.existsByProductId(id)` and `invoiceItemRepository.existsByProductId(id)` - a product referenced by history cannot be deleted (409 Conflict). Referential history stays intact; the shop can stop selling it instead. Categories and suppliers get the same guard through their repositories.

Follow-up: What about soft delete?

Remember: Soft delete (active flag) is the next step - I chose hard delete with guards for simplicity.

e) Database questions (schema, joins, transactions, indexing)

Q51. Walk me through your schema.

Answer: Nine tables: `users`, `category`, `supplier`, `product` (FK category), `purchase` (FK supplier) with `purchase_item`, `stock_batch` (FK product, has `version`), and `invoice` with `invoice_item` (FKs invoice/product/stock_batch). Every shop-scoped table also carries owner_id - the owning shop's user id. Money columns are `DECIMAL(19,2)`, GST rate `DECIMAL(5,2)`. Invoice items keep snapshot columns for name, HSN, price and rate.

Follow-up: Why both purchase and stock_batch?

Remember: Purchase = the document; stock_batch = current sellable stock with expiry.

Q52. Why a separate stock_batch table instead of quantity on product?

Answer: Because the shop needs **per-batch identity**: different purchase dates, cost prices and expiry dates for the same product. FEFO is only possible when you can order batches by expiry, and expiry alerts need the date per quantity unit. A single `product.quantity` cannot express '30 units expire next month'.

Follow-up: How does billing use it?

Remember: FEFO JPQL orders these rows: `ORDER BY expiryDate ASC, id ASC`.

Q53. Explain the FEFO query as SQL.

Answer: JPQL: `SELECT b FROM StockBatch b WHERE b.product.id = :productId AND b.quantity > 0 AND b.expiryDate >= :today ORDER BY b.expiryDate ASC, b.id ASC`. In SQL: `SELECT * FROM stock_batch WHERE product_id = ? AND quantity > 0 AND expiry_date >= CURRENT_DATE ORDER BY expiry_date, id`. The `id` tie-breaker makes ordering deterministic for equal dates.

Follow-up: What index would you add for it?

Remember: Composite index `(product_id, expiry_date)` - equality first, ordering second.

Q54. How do you avoid the N+1 problem?

Answer: Three tools. **JOIN FETCH** in JPQL for known traversals (`findByIdWithItems`, `findAllWithDetails`, `findAllWithBatches`) plus `DISTINCT` where joining a collection multiplies rows. **LAZY** everywhere so list queries do not drag associations. And `open-in-view=false` so any accidental lazy load fails loudly instead of silently adding queries.

Follow-up: What is N+1 exactly?

Remember: 1 query for parents + N queries for children - e.g., 1 for invoices, then 1 per invoice for items.

Q55. How are transactions used in this project?

Answer: Write methods carry `@Transactional`: `InvoiceService.create`, `PurchaseService.create`, `AuthService.register`. Each is an **aggregate-level atomic unit** - bill + items + stock updates commit together or not at all. Reads use `readOnly = true`. Rollback happens on any `RuntimeException`, which is exactly how insufficient stock leaves the database untouched.

Follow-up: What is the default propagation?

Remember: REQUIRED - joins an existing transaction or opens one; correct for these entry points.

Q56. What happens when two users bill the last unit at the same time?

Answer: Both may read quantity 5; each updates its batch row; Hibernate's `@Version` makes the second update match zero rows -> `ObjectOptimisticLockingFailureException` -> my handler returns **409** "Stock was updated by someone else. Please retry." So stock can never go negative from a lost update - the client simply retries.

Follow-up: Could the invoice number duplicate in that scenario?

Remember: Numbers are read-then-write, so a theoretical race exists - acknowledge it; see limitations.

Q57. Which columns are unique and how?

Answer: `users.email` is globally unique in the DB (`@Column(unique = true)`) - duplicates surface as DuplicateKeyException -> 409 from my advice. The shop-scoped ones are per-tenant: category.name, barcode and invoice numbers are checked only inside the caller's own rows (`existsByOwnerIdAndName`, `existsByOwnerIdAndBarcode`, `findTopByOwnerIdOrderByIdDesc`), so two shops may reuse the same names and numbers while a second one inside the same shop is still 409.

Follow-up: Why not a DB unique index on barcode too?

Remember: Fair point - I would add it; app check exists for a friendly message first.

Q58. Is your schema normalized? Any denormalization?

Answer: About **3NF**: dimensions (category, supplier) are separate, transactions reference them by FK, no repeating groups. The deliberate denormalization is **snapshot columns on invoice_item** (name, HSN, price, GST rate) so historical bills remain correct when product data changes - a standard accounting requirement, chosen consciously rather than by accident.

Follow-up: What is the cost of that denormalization?

Remember: Storage duplication and no automatic update - acceptable because history must not change.

Q59. How does H2 differ from MySQL here?

Answer: H2 runs in `MODE=MySQL` compatibility mode, and I kept to portable JPQL plus ANSI date literals, so queries behave the same. Real differences: H2 is embedded/file-based and not for concurrent production load; MySQL needs strict SQL mode care and index tuning. Tests run on in-memory H2; production config points `DB_URL` at MySQL.

Follow-up: Have you run it on MySQL?

Remember: If you have not, say so: "The path is env-var configured and portability was a design goal; I have not load-tested on MySQL."

Q60. Where are the indexes? How would you tune this schema?

Answer: Today: primary keys plus unique constraints (auto-indexed) - no explicit index DDL in the repo since Hibernate generates the schema. Tuning plan: composite indexes `stock_batch(product_id, expiry_date)`, `invoice(invoice_date)`, `invoice_item(invoice_id)`, owner_id composites for tenant scans, FK columns; then `EXPLAIN` the FEFO and report queries on real MySQL data before deciding anything else.

Follow-up: Does every FK need an index?

Remember: MySQL enforces FKs efficiently only with an index on the child FK column - yes, index them.

f) API and backend design questions

Q61. What does your REST API look like?

Answer: About 28 endpoints under `/api`: **auth** register/login (both permitAll); **categories**, **products**, **suppliers** full CRUD; **purchases** POST/GET; **stock** view + `/low` + `/expiring` + `/expired`; **invoices** POST, paged GET, GET by id, GET `/{id}/pdf`; **reports** summary, monthly-sales, top-products, gst-summary. All JSON except the PDF; errors always `{status, message}`.

Follow-up: Why not `/api/v1/...`?

Remember: Versioning: fair gap - I kept one API stable for a portfolio; in production I would version from day one.

Q62. Why are endpoints nouns and not verbs?

Answer: REST convention: HTTP methods carry the verb - `GET /api/products` reads, `POST` creates, `PUT /{id}` updates, `DELETE /{id}` removes. Actions that are not plain CRUD are modeled as sub-resources or filtered reads: `/api/stock/low` (a view of stock), `/api/reports/gst-summary?month=...` (a query), `POST /api/invoices` (a domain operation).

Follow-up: Why is purchase a POST and not PUT?

Remember: It creates a new document - creation is always POST.

Q63. Explain your request/response and error contracts.

Answer: Requests and responses are DTOs: e.g. `InvoiceRequest {customerName, discount, items[{productId, quantity}]}` -> `InvoiceResponse {invoiceNumber, totals, items[...]}`. Lists come back as `PagedResponse {content, page, size, totalElements, totalPages}`. Errors are always `{"status": 400, "message": "..."}` from `GlobalExceptionHandler`, including 401/403 bodies written by the security layer.

Follow-up: Why not a richer error format (code, field, traceId)?

Remember: Richer is better at scale; one shape was right for this scope - say that.

Q64. What status codes do you return?

Answer: 200 read/update, **201** created (`ProductController.create`), **204** no content (delete), **400** validation or insufficient stock, **401** bad/missing token, **403** wrong role, **404** missing resource, **409** duplicates, delete-in-use conflicts and optimistic lock conflicts, **500** generic. The smoke test asserts the key ones.

Follow-up: Why is duplicate email a 409 and not 400?

Remember: 409 Conflict = state conflict with existing resource - precise semantics.

Q65. How do you validate input?

Answer: Two layers. **Declarative** on DTOs with `@Valid`: `ProductRequest` has `@NotBlank name`, `@DecimalMin("0.01") sellingPrice`, `@Min(0) reorderLevel`; `InvoiceRequest.items` is `@NotEmpty` with `@Valid` cascade to items (`@NotNull productId`, `@Min(1) quantity`). **Business rules** in services: negative discount, discount over subtotal, stock availability - thrown as my own exceptions with clear messages.

Follow-up: What happens if validation fails?

Remember: 400 with all field errors joined into one message by the advice.

Q66. How do you handle CORS?

Answer: `SecurityConfig.corsConfigurationSource()` allows exactly `http://localhost:5173` with methods GET/POST/PUT/DELETE/OPTIONS and headers Authorization + Content-Type. In production the frontend origin changes there (or via env). It is a whitelist, not *, so only my UI can call the API with credentials.

Follow-up: Why allow OPTIONS?

Remember: Preflight requests for cross-origin calls - required for Authorization header.

Q67. Why stateless sessions?

Answer: `SessionCreationPolicy.STATELESS` means Spring never creates or stores an HTTP session - every request proves itself with the JWT. Benefits: no sticky sessions at the load balancer, any instance can serve any request, memory per user is zero on the server. Cost: revocation requires token expiry (24 h) or a denylist - a documented limitation.

Follow-up: Does that affect scalability?

Remember: It is the enabler for horizontal scaling - connect it to their scalability question.

Q68. How would you version or evolve this API without breaking clients?

Answer: Practices I would apply: additive changes first (new optional fields), never rename/remove response fields without a version bump; URI versioning (`/api/v1`) when breaking; deprecation headers and a migration window; contract tests so CI fails on accidental breaking changes. Today the repo has a single unversioned API because its only client is my own frontend.

Follow-up: Give an example of a breaking change.

Remember: Changing `grandTotal` to a nested object, or removing `invoiceNumber` - those need v2.

Q69. How do you secure the reports endpoint specifically?

Answer: `/api/reports/**` is explicitly `hasRole("OWNER")` in `SecurityConfig` - STAFF gets 403 even with a valid token. `ReportService` methods are read-only transactions computing summary, monthly sales, top products and GST summary. Tests in `SecurityRulesTest` prove a STAFF token receives 403 there.

Follow-up: Frontend also hides the Reports link for STAFF?

Remember: UI hiding is convenience; the URL rule is the security - say both, in that order.

Q70. Is this API idempotent where it should be?

Answer: GET, PUT and DELETE follow idempotent semantics (repeating them yields the same state). `POST /api/invoices` is **not** idempotent by nature - each bill creates a new record; a client retry after a network timeout could double-bill. In production I would add an idempotency key header that the service deduplicates on before creating the invoice.

Follow-up: How would you implement the key?

Remember: Store key -> invoiceNumber with a unique index; second request returns the first result.

g) Frontend questions

Q71. How is the frontend structured?

Answer: `App.jsx` defines routes: `/login` and `/register` public, everything else inside `ProtectedRoute` -> `Layout` (Sidebar + Navbar + page outlet). Twelve pages live in `pages/` (Login, Register, Dashboard, Billing, Products, Categories, Suppliers, PurchaseNew, Stock, Invoices, Reports, Users); shared UI in `components/` (Modal via portal, Loader, StatCard, Pagination); API in `api/axios.js`; auth state in `context/AuthContext.jsx`. If the backend is unreachable, the Axios interceptor serves a mock dataset instead - an offline demo mode flagged by a navbar badge.

Follow-up: Why is Modal portaled to body?

Remember: The CSS transform containing-block bug - a real fix you can explain with a diagram.

Q72. How does state flow in the app?

Answer: Global: only auth - `AuthContext` exposes `{token, user, isOwner, login, logout}` from localStorage. Everything else is page state with hooks: the billing cart, search filters, the selected invoice for the modal, active stock tab. Data fetching happens in `useEffect` per page against the Axios client. No Redux, no global store for data - each page owns its own.

Follow-up: What if two pages need the same data?

Remember: Today they refetch - fine at this scale; next step would be a shared query cache (React Query).

Q73. How does the billing screen work?

Answer: The clerk searches products (GET `/api/products?search=`), clicks to add lines to a cart with quantities, optionally sets an invoice-level discount and customer name, then Save POSTs `{items, discount}` to `/api/invoices`. On success the page switches to a saved-bill view showing the invoice number, totals, and a **Download PDF** action that fetches the PDF as a blob with the JWT.

Follow-up: What if the server rejects (400 insufficient stock)?

Remember: The response message renders in an alert; cart stays intact so the clerk can adjust - good UX detail.

Q74. How does the frontend handle errors?

Answer: Axios response interceptor catches **401** globally: clears token/user, redirects to `/login`. All other errors go through `errorMessage(error)`: it prefers `error.response.data.message` from my backend contract, detects a dead server ("Cannot reach the server. Is the backend running?"), and falls back to a generic line - rendered in styled alert boxes on each page.

Follow-up: And 403?

Remember: 403 shows the server's permission message in the same alert - no special redirect.

Q75. How did you make the UI look custom, not template-y?

Answer: I built a small **design system**: an ink/brand/gold palette and Plus Jakarta Sans font defined in `tailwind.config.js`, reusable classes (`.card`, `.btn`, `.input`, `.badge`, `.table-*`) in `index.css`, a dark grouped sidebar with a hand-drawn SVG brand tile, and entrance animations with `fadeUp`/`popIn`. Consistency comes from the classes; the look comes from my palette.

Follow-up: Any design lessons from the redesign?

Remember: The modal bug came from animations - show that design and correctness interact.

Q76. How does route protection work in React?

Answer: `ProtectedRoute` reads `token` from `AuthContext`; if absent it returns `<Navigate to="/login" replace />`, else it renders `<Outlet/>` for child routes. The server is still the real gate - a STAFF token can open `/products` but write APIs return 403. `replace` keeps the login route out of history so Back does not bounce endlessly.

Follow-up: Is client-side protection security?

Remember: Say clearly: UI guard = UX, server rules = security. Never conflate them.

Q77. How do charts work and how do you keep the bundle small?

Answer: Recharts renders the monthly sales line and top-products bar charts from `/api/reports/*` JSON. To keep the initial load fast, `vite.config.js` defines `manualChunks` so **Recharts and React are separate cached chunks** - main app code is ~175 kB, React ~164 kB, Recharts ~384 kB (gzip 49/53/106 kB). Routes could be lazy-loaded next.

Follow-up: What does code splitting actually buy?

Remember: Repeat visits download only changed chunks - faster loads and better caching.

Q78. How does the frontend get the PDF with authentication?

Answer: Not by a plain link - a link cannot attach the Authorization header. The page does `axios.get(/invoices/{id}/pdf, {responseType: 'blob'})`, receives binary data, creates an object URL and clicks a hidden anchor to download. `Content-Disposition`-style filenames are handled from the invoice number so files are named `INV-2026-0001.pdf`.

Follow-up: What if the token expires mid-click?

Remember: Interceptor 401 handler kicks in - user lands on login, cart state on billing is the only loss.

h) Security questions

Q79. How are passwords stored?

Answer: **BCrypt** hashes with per-user salt - `BCryptPasswordEncoder` bean in `SecurityConfig`, applied in `AuthService.register()` via `passwordEncoder.encode(...)`. Login verifies with `AuthenticationManager` (which uses the encoder). Plain text never touches the DB, and hashes never leave it - no endpoint returns a password.

Follow-up: Why BCrypt over SHA-256?

Remember: Salted + intentionally slow (cost factor); plain SHA is fast to brute-force and unsalted.

Q80. Explain your JWT in detail.

Answer: HS256-signed token, 24-hour expiry, `sub` = email, custom claims `role` and `name`. Secret comes from `JWT_SECRET` via `@Value("${app.jwt.secret}")` - config, not code. `JwtUtil.isValid()` verifies signature, subject match, and expiry. It is **stateless**: nothing is stored server-side, so any instance can validate any request.

Follow-up: What is in the payload - is it encrypted?

Remember: JWT payload is base64, NOT encrypted - anyone can read claims; signature only prevents tampering.

Q81. What if someone steals the token?

Answer: They are that user for up to 24 hours - that is the honest answer. Mitigations in the design: short-ish expiry, HTTPS in production, and the frontend catching 401 to force re-login. What I would add: refresh-token rotation with server-side revocation, and moving access tokens out of localStorage into httpOnly cookies to reduce XSS theft.

Follow-up: How do you revoke early today?

Remember: You cannot, cleanly - "rotate JWT_SECRET" is the nuclear option; admit it.

Q82. Is the app vulnerable to CSRF? To XSS?

Answer: **CSRF**: low risk - auth travels in an `Authorization` header, not cookies, and CORS allows only my frontend origin; I disabled CSRF deliberately for that stateless model. **XSS**: token in localStorage is readable by injected script - my main exposure; mitigations would be CSP headers, output escaping discipline (React escapes by default), and eventually httpOnly cookies.

Follow-up: React escapes by default - does that make XSS impossible?

Remember: `dangerouslySetInnerHTML` or URL-based injection can still hurt - say "reduces, not eliminates".

Q83. How does role authorization work technically?

Answer: The JWT carries `role`; `UserDetailsServiceImpl` loads the user and maps it to `ROLE_OWNER`/`ROLE_STAFF` authorities; `SecurityConfig` URL rules use `hasAnyRole`/`hasRole`. `anyRequest().hasRole("OWNER")` makes everything else owner-only. Denials hit my custom access-denied handler -> 403 JSON. Five tests in `SecurityRulesTest` lock this behavior in.

Follow-up: What if role is removed from a user while a token is live?

Remember: Token still carries old role until expiry - the loadUserByUsername path refreshes it when a DB lookup happens; acknowledge the 24-h window.

Q84. How do you protect against SQL injection?

Answer: All queries go through JPA: JPQL with **bound parameters** (`@Param`), derived query methods, and criteria-style construction - no string-concatenated SQL anywhere in the repo. User input never becomes SQL text. `search` uses `LIKE :search` as a bound parameter as well.

Follow-up: What about LIKE wildcards in search?

Remember: User-typed % acts as a wildcard - cosmetic annoyance, not injection; could escape it.

Q85. How are secrets managed?

Answer: Nothing secret is hard-coded in Java. `application.properties` uses `${JWT_SECRET:local-dev-default}` - environment variables win; the dev default is obviously non-production and called out. DB credentials follow the same pattern (`DB_USERNAME`, `DB_PASSWORD`). `.gitignore` blocks `.env` files so secrets cannot be committed.

Follow-up: Is the dev default safe?

Remember: It is fine ONLY for local; production must set JWT_SECRET - I would also warn in README.

Q86. What does the register endpoint do about privilege escalation?

Answer: Three modes in `AuthService.register()`: an anonymous caller may register only as OWNER - that starts a brand new shop (the old first-user-bootstrap rule is gone); an OWNER token may register only STAFF, and the new row gets that owner's owner_id; a STAFF token always gets 403. Duplicate email is 409, an unknown role string is 400. Nothing lets anyone write into an existing shop they do not own.

Follow-up: Can an anonymous user or a STAFF create users?

Remember: Both get 403: a STAFF token is never the owner, and anonymous registration with role STAFF is refused with "Register as an owner to start a new shop".

Q87. How do you handle unauthorized access on the frontend?

Answer: Axios response interceptor: any 401 clears `token` and `user` from localStorage and hard-redirects to `/login` (guarded to avoid loops). `ProtectedRoute` sends token-less users to login before rendering. `isOwner` also hides owner-only nav items - convenience only; the server still enforces truth.

Follow-up: What if the API returns 403 - are they logged out?

Remember: No - 403 shows a permission message; logout is only for lost authentication.

Q88. What security testing have you done?

Answer: Behavioral tests: `SecurityRulesTest` covers STAFF allowed/forbidden paths, OWNER-only reports, unauthenticated 401; `AuthControllerTest` covers the five register rules and wrong-password 401; TenantIsolationTest proves two owners cannot read each other's data (cross-tenant 404s, per-shop category names, scoped reports); the smoke script asserts 401/403 and that expired stock cannot be sold. I have **not** run OWASP ZAP or penetration testing - that is on my to-learn list for production readiness.

Follow-up: What would you test first with ZAP?

Remember: Auth endpoints, IDOR on invoice ids, and rate limiting on login.

i) Scalability and performance questions

Q89. How would you scale this to 100,000 users?

Answer: First, what already helps: **stateless JWT** means I can run N identical jar instances behind a load balancer with no session affinity. Then: MySQL with the indexes I listed, read replicas for reports, Redis for hot product search and dashboard summary, PDF generation moved to a queue, frontend on a CDN, and rate limiting at the edge. Multi-tenancy already exists - `owner_id` on every shop-scoped table - so sharding by `owner_id` is a data move, not a redesign.

Follow-up: What breaks first?

Remember: Report queries and connection count - name them concretely.

Q90. Where are the performance bottlenecks today?

Answer: Three: **report endpoints** loop over full invoice lists in Java (O(n) memory - fine for thousands of bills, bad for millions); **FEFO query** without a `(product_id, expiry_date)` index scans batches per product; **top-products** loads 30 days of invoice items. Pagination keeps lists bounded, but these aggregations are the first place I would rewrite as SQL with EXPLAIN checks.

Follow-up: How would you rewrite monthly sales in SQL?

Remember: `GROUP BY DATE_FORMAT(invoice_date, '%Y-%m')` with sum/grand_total - then cache it.

Q91. Do you use caching?

Answer: Not yet - there is no `@Cacheable`, Redis, or CDN in the repo, and I will not claim otherwise. The natural first cache targets: product list/search (changes rarely, read-heavy), dashboard summary, and monthly GST reports (immutable once a month closes). I would use Caffeine for single-instance or Redis once multiple instances exist.

Follow-up: What invalidation strategy?

Remember: Cache aside with explicit invalidation on write; time-bound TTL for reports.

Q92. How do you handle many rows in list endpoints?

Answer: Server-side pagination everywhere it matters: products and invoices expose `page`/`size` (size capped at 100) and return `PagedResponse` with totals. Invoice list uses a **constructor projection** so line items are never loaded. The frontend renders tables with a Pagination component fetching one page at a time.

Follow-up: Deep pagination (page 10000) - problem?

Remember: OFFSET gets slow - keyset/seek pagination by id > lastId is the fix. Good bonus answer.

Q93. Is your connection pool tuned?

Answer: Spring Boot's default **HikariCP** pool applies (I did not override pool sizes in `application.properties`). For production I would size it from `DB max connections / instance count`, set validation timeouts, and monitor pool exhaustion - honestly, it is unconfigured today, and defaults are correct for dev scale.

Follow-up: What is the default pool size?

Remember: Hikari default max is 10 - good number to remember.

Q94. How does optimistic locking behave under high contention?

Answer: On hot rows (same batch sold repeatedly) retries pile up: every conflict is a 409 asking the client to retry. At shop scale that is rare; at high contention the alternative is pessimistic `SELECT FOR UPDATE` for the deduction read, or aggregating stock into a single counter row with atomic `UPDATE ... SET qty = qty - ?` and a check constraint.

Follow-up: Which would you pick for flash-sale load?

Remember: Atomic UPDATE with quantity >= ? guard - contention-safe without retry storms.

Q95. How would you make reports production-grade?

Answer: Move aggregations to SQL with proper indexes; materialize daily rollups in a `sales_daily` table written during billing (pre-aggregation); serve charts from rollups instead of raw invoice items; cache the monthly GST summary after month close. Keep Java loops only for small datasets where readability wins.

Follow-up: Does pre-aggregation hurt correctness?

Remember: Rebuildable from source + nightly reconciliation = standard approach.

Q96. Frontend performance - what have you done?

Answer: Manual chunking splits **Recharts** and **React** into cached vendor chunks (visible in the build output), pages fetch only what they render, and tables paginate. Not done: route-level lazy imports, image optimization (few images anyway), and virtualized long lists. Bundle is ~175 kB main + vendors - healthy for this app.

Follow-up: What is first contentful paint for your app?

Remember: Do not invent numbers - say you have not measured it and would use Lighthouse.

j) Challenges, bugs, failures and learnings

Q97. What was the hardest part of the project?

Answer: **Billing correctness**: four concerns - discount split, GST rounding, FEFO deduction, invoice numbering - all inside one transaction, with money that must add up to the paisa. The hard part was making rounding deterministic: divide at scale 8, round at 2, give the remainder to the last line, and split CGST/SGST as `gst/2` and `gst - cgst` so nothing drifts.

Follow-up: How did you prove it was correct?

Remember: Smoke test asserts exact values: 190 x 12% = 22.80 -> 11.40 + 11.40.

Q98. Describe a real bug you fixed.

Answer: After the UI redesign, the **invoice detail modal was clipped** to a strip of the page. Root cause: the page wrapper's CSS animation ended at `transform: translateY(0)` with fill-mode `both` - an identity transform still makes that div the containing block for `position: fixed`, trapping the modal. Fix: end keyframes at `transform: none` **and** portal the Modal to `document.body`. Verified with screenshots.

Follow-up: How did you find it so precisely?

Remember: Screenshot showed the backdrop clipped exactly at the wrapper's bounds - geometry pointed to containing-block, not z-index.

Q99. A bug that taught you something about testing?

Answer: My MockMvc tests **shared one H2 database** and leaked the logged-in user across tests because `SecurityContextHolder` is thread-local - test 3 passed with test 2's identity. Fix: `@Transactional` rollback on every test class plus `SecurityContextHolder.clearContext()` in `@AfterEach`. Lesson: test isolation is not automatic; framework state lives on threads.

Follow-up: Why not a fresh DB per test?

Remember: Rollback is faster than recreate - trade-off I chose deliberately.

Q100. What failed during integration and how did you recover?

Answer: The end-to-end smoke script initially failed on a second run: it creates a product each run, so seed-count assertions broke against a dirty database. I diagnosed it as **non-idempotent test data**, fixed the script's assumptions, and documented 'run against a fresh DB' (delete `backend/data`, restart). After that: all 62 checks green. Lesson: test fixtures must be explicit about their starting state.

Follow-up: Would you make the script idempotent?

Remember: Yes - unique suffixes or cleanup phase; at the time fresh-DB was simpler and documented.

Q101. Tell me about the git incident.

Answer: `git` suddenly reported 'your current branch appears to be broken' - `refs/heads/master` contained zero bytes instead of a commit SHA. Since the objects were intact, I repaired the ref by writing the last known commit hash back into the file and verified with `git log`. No history was lost. I documented the incident in PROGRESS.md rather than hiding it.

Follow-up: What caused it?

Remember: Be honest: "I could not reproduce the exact cause - uncertain, recorded as such."

Q102. What is the biggest mistake in your code?

Answer: Probably **`nextInvoiceNumber()` reading the last invoice to increment** - elegant for one counter, racy for many. I knew the trade-off when writing it (documented in README limitations), chose the simple version per shop - isolation by owner_id landed later, but two staff billing simultaneously in the same shop can still collide - and I would replace it with a yearly sequence row locked with `FOR UPDATE` before real production use.

Follow-up: Why not fix it now?

Remember: Scope discipline is a valid answer - but show you know the exact fix.

Q103. What would you do differently from the start?

Answer: Three things: add **Docker and a CI pipeline** from commit one (I added them last, i.e., not at all - CI is still missing); write **tests alongside each feature** instead of a dedicated test phase (less context switching, earlier feedback); and design **multi-tenancy** into the schema from commit one - I actually retrofitted it mid-project (`owner_id` across six tables, every query tenant-scoped), which worked but proved exactly why it belongs up front.

Follow-up: Any design-level change?

Remember: Mention report queries - SQL aggregation instead of Java loops if I had expected scale earlier.

Q104. How did you verify your work end to end?

Answer: Three layers: `mvn clean test` (34 automated tests) for logic; `scripts/smoke-test.ps1` (62 checks) against the running server for real HTTP behavior - FEFO order, exact GST amounts, rollback, role rules, PDF bytes; and manual browser verification with screenshots for the frontend, including the modal fix. PLAN.md required a green verification after every step.

Follow-up: Any checks you skipped?

Remember: Frontend has no automated tests - honest gap, next step Playwright.

Q105. What did debugging FEFO look like?

Answer: The risk was selling the wrong batch. I wrote the query to order by `expiryDate ASC, id ASC`, then proved behavior with a scenario: two Oil 1L batches (B1 earlier expiry). The smoke script sells across them and asserts **B1 drains to 0 before B2 decrements**. Seeing the exact quantities flip in the assertions gave me confidence no loop-boundary bug existed.

Follow-up: What if two batches share an expiry date?

Remember: `id ASC` tie-breaker makes it deterministic - oldest record first.

k) Managerial and behavioral questions tied to the project

Q106. How did you plan and manage this project yourself?

Answer: I kept three living documents: **PLAN.md** (ordered build steps, each checked only after verification), **DECISIONS.md** (every trade-off with rationale, so I would not re-litigate choices), and **PROGRESS.md** (what ran, what the output was - including failures). Weekly I reviewed what was blocked and re-ordered. It mimics how I would work in a team without a scrum master.

Follow-up: How do you decide what to build next?

Remember: Dependencies first (schema before UI), then risk (billing core early), then polish (redesign last).

Q107. You are joining a team mid-project. What do you do in week one?

Answer: Read the README and architecture docs, run the build and tests locally until green, then trace one real feature end to end through the code - for this repo I would pick billing: controller, service, FEFO query, tests. Then fix a small real issue to touch the whole loop, and ask my buddy about the two decisions I cannot infer from code (deployment, monitoring).

Follow-up: What if the tests do not pass on your machine?

Remember: Environment first (JDK, DB, env vars), then escalate with the exact error - never silently skip tests.

Q108. Tell me about a time you made a decision without all the information.

Answer: Invoice numbering: I had to choose between a DB sequence and the read-and-increment approach before knowing the deployment scale. I picked read-and-increment - one shop, one counter - **documented the concurrency caveat in README**, and defined the upgrade path. Decision-making under uncertainty means choosing, recording, and keeping the door open.

Follow-up: What if they told you 50 counters from day one?

Remember: Then sequence-per-year with row locking - say the alternative exists, not that you guessed.

Q109. How do you handle conflicting advice (e.g., on JWT vs sessions)?

Answer: I state the constraint first, then decide. For this app: single service, stateless scaling goal, no existing identity provider - JWT fit. If a senior argued sessions for revocation, I would ask what changes (ops model, instance count), then either switch or add refresh-token revocation. Conflicts resolve faster when you debate constraints instead of preferences.

Follow-up: What if they overruled your choice?

Remember: Commit fully to the team's decision, keep the rationale in ADR - no silent resentment.

Q110. How did you prioritize features?

Answer: Backend value chain first: auth -> CRUD -> purchases/stock -> billing (the differentiator) -> reports -> frontend -> tests -> docs -> polish. Money features got the deepest correctness work; nice-to-haves (UI redesign, animations) came last so they never blocked learning the domain. The smoke script defined 'done' for each slice.

Follow-up: What did you cut?

Remember: Payments/credit, returns, IGST - named cuts are credible; pretending nothing was cut is not.

Q111. How do you ensure quality without a QA team?

Answer: I defined quality as checks I can run: 34 unit/MockMvc tests for logic, a 62-check smoke script for real HTTP flows (including failure cases like insufficient stock and expired batches), a green `npm run build`, and a manual browser pass with screenshots for UI. 'Verified by command output' was a rule in my plan - proof, not vibes.

Follow-up: What is the weakest quality link?

Remember: Frontend automated tests - say Playwright as the plan, plus load testing as missing.

Q112. A stakeholder wants a feature that breaks your architecture (e.g., editing past invoices). What do you do?

Answer: First clarify the need: is it correction-by-credit-note or true mutation? GST practice favors credit notes, which fits my immutability design (snapshots, historical integrity). If they truly need edits, I would scope it: an OWNER-only adjustment flow with audit trail, implemented as new invoice rows, not mutation. I show the cost, then propose the safest path that meets the need.

Follow-up: What if they insist on direct edit?

Remember: Escalate the risk (audit, GST filing mismatch), decide together, document it.

Q113. How did you handle scope creep on your own project?

Answer: Every nice idea went to a 'next phase' list instead of the active plan - payments, returns and React tests are all recorded in limitations - multi-tenancy I later promoted from that list into the active plan. When I redesigned the UI, it was a bounded commit after the core was green, not a mid-feature rewrite. Writing the scope down is what stops a solo developer from endlessly polishing.

Follow-up: Any feature you added too late?

Remember: Docs/INTERVIEW_NOTES - honest answer: speaking about the project was as important as building it.

Q114. What would you do differently if you rebuilt it in a team?

Answer: Contract-first API (OpenAPI) so frontend and backend work in parallel; feature branches with PR reviews instead of straight-to-master; CI running tests and smoke on every push; and shared ownership of DECISIONS as ADRs. Solo, my speed came from owning everything; in a team the goal shifts to making decisions visible to others.

Follow-up: How comfortable are you with code review?

Remember: Ask for it actively - 'I would rather get comments early' shows maturity.

Q115. What is your biggest learning overall?

Answer: That **correctness is a design activity, not a testing activity**. The FEFO, rounding and rollback behaviors were decided in the design (transaction boundaries, one rounding utility, snapshot columns); tests then proved them. Debugging a wrong design is far costlier than writing the careful version up front - and documenting the why is what makes it survive future changes.

Follow-up: Give an example where a test caught something design missed.

Remember: Test isolation leak (SecurityContext) - framework behavior, not domain design - still a great example.

l) Tricky / cross questions (and handling "I don't know")

Q116. Isn't this just CRUD with a billing feature?

Answer: The CRUD part is deliberately plain - that is the boring 40%. The value is in the transactional core: FEFO deduction across batches, discount distribution with exact rounding, GST split, invoice numbering and rollback semantics, plus concurrency protection with `@Version`. CRMs have CRUD too; what distinguishes this project is that money and stock stay correct under failure.

Follow-up: Where exactly is it not CRUD?

Remember: Point to `InvoiceService.create()` - one method, four invariants.

Q117. You say stateless auth - so how do I log out a user?

Answer: Today 'logout' is client-side: the token is deleted from localStorage - it remains cryptographically valid until its 24-hour expiry. That is a real limitation, not a feature. The production design is refresh-token rotation with a server-side revocation list, or a short access token (minutes) plus refresh. I documented this in README limitations rather than glossing over it.

Follow-up: Can I revoke faster right now?

Remember: Changing JWT_SECRET invalidates all tokens - a blunt emergency tool, mention it as such.

Q118. Your reports load everything into Java. That is bad practice, right?

Answer: At shop scale (thousands of invoices) it is simple and portable - a conscious trade-off I documented. At millions of rows it becomes the wrong tool: I would move to SQL `GROUP BY`, add indexes on `invoice_date`, pre-aggregate daily rollups, and cache month-close GST summaries. The key is knowing the scale where the current approach breaks - mine is well before production traffic.

Follow-up: How would you migrate?

Remember: Keep the response DTOs identical - swap the internals, tests stay green.

Q119. What does `spring.jpa.defer-datasource-initialization=true` do?

Answer: Normally `data.sql` runs when the datasource is created - before Hibernate has made the tables. This flag defers SQL init until **after** Hibernate has applied the schema, so my seed script finds its tables. It is exactly why the idempotent `INSERT ... SELECT ... WHERE NOT EXISTS` seed works on every startup with `ddl-auto=update`.

Follow-up: What does SEED_MODE=never do?

Remember: `spring.sql.init.mode=${SEED_MODE:always}` - env var switches seeding off for production.

Q120. How would you answer a question you do not know?

Answer: Honest structure: "I have not worked with X. My current understanding is A, based on B. To verify I would check the official docs and write a small spike - like I did when jjwt's `.build()` method bit me." Then steer back to something adjacent I do own. Confidence is not knowing everything; it is knowing how you would find out.

Follow-up: Example of something you do not know?

Remember: Pick a real gap (e.g., Testcontainers) - honesty beats bluffing to a sharp interviewer.

Q121. Why is your JWT secret default in the repo?

Answer: Because the default is a development placeholder that lets `mvn spring-boot:run` work with zero setup, and production overrides it via `JWT_SECRET` - the property is `${JWT_SECRET:default}`. Shipping an app that will not boot without env vars hurts the clone-and-run experience. The README and comments mark it as change-before-production. For a real product I would fail startup if the secret is unset.

Follow-up: Isn't a committed secret always wrong?

Remember: The line: committed **placeholder** for dev is acceptable; a committed **production** secret never is.

Q122. Delete a product - what happens if it has invoice items?

Answer: It does not delete: `ProductService.delete()` checks `existsByProductId` on purchase items and invoice items first and throws `ConflictException` -> 409 "Product is used in an invoice and cannot be deleted". Financial history must stay intact. The shop can rename the product or mark it inactive instead.

Follow-up: So how do you ever remove a discontinued product?

Remember: Today: only truly unused products; a `discontinued` flag would be the proper feature - name it as future work.

Q123. Could Hibernate's automatic schema update damage a production DB?

Answer: Yes - `ddl-auto=update` adds columns/tables but will not drop anything, and it is not a migration tool. For production the right answer is `validate` plus versioned migrations (Flyway/Liquibase) with reviewed SQL. The current default favours dev convenience; I list moving to Flyway among my rebuild changes.

Follow-up: What is Flyway exactly?

Remember: Versioned SQL files (V1__init.sql ...) applied in order with a history table.

Q124. Two of your own docs disagree - DECISIONS said X but code does Y. Which is right?

Answer: The **code** is the truth; the document gets corrected. That actually happened: DECISIONS.md described `data.sql` inserts in a way that did not match the idempotent `INSERT ... SELECT ... WHERE NOT EXISTS` form I wrote, and I fixed the document in the docs commit. Documentation drift is normal; the discipline is to reconcile it rather than explain it away.

Follow-up: How do you prevent drift?

Remember: Docs changes in the same commit as code changes - good habit to state.

m) Scenario questions (what-if)

Q125. 10,000 users hit GET /api/products at once - what happens?

Answer: Each request is a paged query - cheap individually, but 10,000 concurrent ones exhaust the default Hikari pool (~10 connections) and the queue grows. Mitigations in order: edge rate limiting, CDN/cache for the product list (it reads well), connection pool sizing, read replicas, and horizontal app instances - which stateless JWT already permits. Today I would honestly expect queuing, not graceful load shedding.

Follow-up: What is the first thing you would monitor?

Remember: Hikari pool active/pending + p95 latency - name metrics, not vibes.

Q126. The database goes down at 6 PM (billing hours). What do you do?

Answer: Immediate: stop writes cleanly - my API returns 500 generic messages, the frontend shows 'Something went wrong'; I would alert on connection failures and check the DB server. Recovery: failover to replica if configured, restore from backup, verify with the smoke script. Prevention for the future: proper MySQL HA (primary-replica with automatic failover), connection retry with backoff, and monitoring. (No backup/replica setup exists in the repo today - that is the honest gap.)

Follow-up: Data lost if primary dies?

Remember: Without binlog/replica: up to the last backup - so scheduled backups + point-in-time recovery are the answers.

Q127. What if two cashiers save bills simultaneously on the same last unit?

Answer: Both transactions deduct the same batch; `@Version` makes the loser's UPDATE match zero rows -> `ObjectOptimisticLockingFailureException` -> my handler returns **409** "Stock was updated by someone else. Please retry." Stock never goes negative; the cashier retries and sees an accurate 'insufficient stock' message if the unit is gone.

Follow-up: User experience of that retry?

Remember: Frontend alert shows the server message - cart intact, clerk adjusts quantity.

Q128. What if a product's GST rate changes from 12% to 18% tomorrow?

Answer: New bills compute at 18% because GST rate is read from the product at billing time. **Old invoices stay correct** because invoice_item stores a snapshot `gst_rate` (and price, name, HSN) per line - historical PDFs and the GST summary keep showing 12%. That snapshot design is exactly why changing master data is safe.

Follow-up: Does the product table update cascade to invoices?

Remember: Never - by design. History is immutable in this schema.

Q129. What if the server and client disagree on money by 0.01?

Answer: They do not compute independently: the **server is the single source of truth** - the client sends items and discount, and every total in the response comes from backend BigDecimal math with HALF_UP to 2 dp. The frontend only formats (`formatMoney` with en-IN locale). Display rounding can never alter stored values.

Follow-up: What if the frontend rounds differently for display?

Remember: Cosmetic only; DB and PDF both come from the same backend computation.

Q130. How do you deploy a new version with zero downtime?

Answer: Build jar, run tests + smoke in CI, then rolling deployment: start the new instance, health-check it, shift load balancer traffic, drain and stop the old one. Stateless JWT makes instances interchangeable - no session migration. Schema changes must be backward compatible (additive first) because old and new versions run together briefly. (None of this exists in the repo yet - it is the plan.)

Follow-up: What about the DB migration during the rollout?

Remember: Expand-contract pattern: add columns -> dual-write/read -> clean up later.

Q131. Someone uploads a product with GST rate 900% - what happens?

Answer: `ProductRequest` rejects impossible input: `@DecimalMin("0")` catches negatives, `@Size(max=20)` bounds codes; 900 itself passes validation as a number - so the service/business layer should bound it (honest gap: no upper clamp today; GST rates in India are 0/5/12/18/28). I would add a rate whitelist or max-100 check, plus a friendly message - defense in depth at the DTO and service layers.

Follow-up: So is that a bug?

Remember: Name your own gaps before they do - then show the one-line fix.

Q132. How would you migrate from H2 to MySQL in production?

Answer: No code change - set `DB_DRIVER`, `DB_URL`, `DB_USERNAME`, `DB_PASSWORD` env vars (the datasource is `${DB_URL:default}`). Steps I would actually take: run schema via migrations instead of `ddl-auto`, export/transform seed and data (H2 `SCRIPT` or keep `SEED_MODE` for demo rows), run the smoke suite against MySQL, then load test. The portability work - MODE=MySQL, portable JPQL - exists for exactly this switch.

Follow-up: What would probably break?

Remember: Date functions and SQL modes in any hand-written SQL - here JPQL keeps that surface small.

n) Multi-tenancy and shop isolation

Q133. You call this multi-tenant - what does that actually mean in the code?

Answer: Every shop-scoped table carries an `owner_id` column (users, category, product, supplier, purchase, invoice). A tenant is one owner: an OWNER's shop id IS their own user id, and every STAFF row stores the owner's id. Services read the caller's tenant with `CurrentTenant.id()` and pass it into every repository query, so isolation lives in the data layer - not in UI hiding.

Follow-up: Why owner id instead of a separate shop table?

Remember: The owner user row IS the shop - one less join, one source of truth; staff simply reference it.

Q134. How does the server know which shop a request belongs to?

Answer: `UserDetailsServiceImpl` builds `AppUserDetails` with tenantId = ownerId != null ? ownerId : id - the JWT names the user, the principal carries the tenant. `CurrentTenant.id()` reads it from the SecurityContext on every call and throws `UnauthorizedException("Please login first")` if anonymous. The tenant is derived from identity, never from a client-supplied header, so it cannot be spoofed.

Follow-up: Why not pass the shop id from the frontend?

Remember: Identity-derived tenant - the frontend never chooses the shop, so tampering is impossible.

Q135. What stops me from reading another shop's invoice by id?

Answer: The owner-scoped query. `findByIdWithItems(id, CurrentTenant.id())` loads invoice AND tenant together; a wrong-shop id comes back empty and the service answers 404 "Invoice not found with id 123" - byte-identical to a genuinely missing row, so enumeration leaks nothing. CategoryService does the same with an explicit owner check after load.

Follow-up: Is every read path like this?

Remember: Yes - products, stock batches (b.product.ownerId), purchases, reports, invoices; TenantIsolationTest pins each one.

Q136. How does registration work once the app is multi-tenant?

Answer: Three modes in `AuthService.register()`: anonymous -> role forced to OWNER, owner_id null (starts a new shop; asking for STAFF is 403 "Register as an owner to start a new shop"); OWNER token -> may only create STAFF with owner_id set to the caller's shop (asking for OWNER is 403 "An owner can only add staff users"); STAFF token -> 403 always. Duplicate email is 409. The old first-user-bootstrap rule is gone: anyone can open their own shop, nobody can write into someone else's.

Follow-up: So an owner token can never create a second owner?

Remember: Never - a second owner only exists as an anonymous sign-up that starts their OWN shop.

Q137. Which columns are unique now that data is shared?

Answer: `users.email` stays globally unique - login is by email. Everything shop-scoped became per-shop: category.name via `existsByOwnerIdAndName`, barcode via `existsByOwnerIdAndBarcode`, and invoice numbers are per-tenant - `nextInvoiceNumber(ownerId)` reads only that shop's last invoice, so INV-2026-0001 exists once per shop, not once globally.

Follow-up: Why not composite unique indexes on (owner_id, name)?

Remember: Service checks give friendly 409s today; composite unique constraints are the production hardening - name it before they do.

Q138. How did you test the isolation?

Answer: `TenantIsolationTest` - six MockMvc tests with two owners: a fresh owner starts empty while the seeded owner keeps their data; owner B gets 404 on owner A's invoice; billing against another shop's product is 404; the same category name is legal in two shops but rejected twice in one; staff created by an owner see only

that owner's shop; reports return only the caller's numbers. The smoke script adds the register rules - 62 checks in total.

Follow-up: You retrofitted tenancy - what was the hard part?

Remember: Every repository method grew an owner_id parameter; CurrentTenant on the principal is the seam that left controllers untouched.

Q139. What was the hardest part of adding multi-tenancy?

Answer: Finding every implicit assumption that ids are global. A single unscoped `findById(id)` anywhere would have leaked across shops, so I changed signatures instead of adding if-checks: repository methods take the tenant as a required parameter, which means a new query cannot be written without deciding whose rows it reads. The suite failed loudly until every path was scoped.

Follow-up: How do you keep it from regressing?

Remember: Review invariant: a repository call without owner_id does not merge - grep proves it, the test suite enforces it.

Q140. Where would you go next with tenancy?

Answer: Three steps: composite indexes on (owner_id, ...) so tenant scans stay fast as rows grow; DB unique constraints on (owner_id, name) backing the service checks; and created_by columns so staff actions inside a shop are attributable. If one database becomes the limit, sharding by owner_id is a data move - the column and every query already exist.

Follow-up: Schema-per-tenant instead?

Remember: Only for noisy neighbours - it costs migrations and connections; at shop scale row-level tenancy is the right trade.

13. Cheat Sheet - One-Page Rapid Revision

Stack in one line each

- Backend: **Java 17 + Spring Boot 3.3.5 + Maven**; web/data-jpa/security/validation starters.
- DB: **H2 file DB default, MySQL via 4 env vars**; schema from Hibernate `ddl-auto=update`; seed via idempotent `data.sql`.
- Auth: **stateless JWT (jjwt 0.11.5, HS256, 24h)** + BCrypt + `JwtAuthFilter`; roles OWNER/STAFF; every query scoped by owner_id.
- PDF: **OpenPDF 1.3.43** (lowagie), amount-in-words in Indian format.
- Frontend: **React 18 + Vite 5 + Tailwind 3.4 + React Router 6 + Axios + Recharts + lucide-react**; no Redux - one AuthContext.
- Quality: **34 tests in 9 classes** + **62-check smoke script**; `npm run build` clean; docs README/DECISIONS/PROGRESS/INTERVIEW_NOTES.

Five flows in one line each

- **Login**: `POST /api/auth/login` -> BCrypt verify -> JWT (claims role+name) -> axios interceptor sends Bearer on every call.
- **Authorize**: `JwtAuthFilter` fills SecurityContext -> URL rules: STAFF gets GET products/categories/stock + GET/POST invoices; everything else OWNER; every query carries the caller's owner_id.
- **Bill**: validate -> split discount (remainder last line) -> per-line GST (cgst=gst/2, sgst=gst-cgst) -> FEFO deduct (ORDER BY expiry) -> invoice number -> cascade save; **all one @Transactional**.
- **Purchase**: supplier check -> per line save PurchaseItem + one new StockBatch -> total -> cascade save.

- **PDF**: protected `GET /api/invoices/{id}/pdf` -> `PdfService` builds Document in memory -> blob download in frontend.

Key numbers (memorize)

```
Spring Boot 3.3.5 / Java 17 / jjwt 0.11.5 / OpenPDF 1.3.43
React 18.3.1 / Vite 5.4.8 / Tailwind 3.4.13 / Recharts 2.12.7
Entities: 9    Endpoints: ~28    Test classes: 9    Tests: 34    Smoke checks: 62
JWT expiry: 24 h (86400000 ms)      Money: BigDecimal, scale 2, HALF_UP
Invoice no: INV-YYYY-NNNN           GST: cgst = gst/2, sgst = gst - cgst
Seed: 20 products, 3 suppliers, 6 categories, 20 batches, 2 invoices
Ports: backend 8080, frontend 5173  Accounts: owner@munimji.local / owner123
```

Top 16 questions to rehearse

- 1. Tell me about your project (2 minutes).
- 2. Why did you choose this stack?
- 3. Walk me through the request lifecycle of `POST /api/invoices`.
- 4. What is FEFO and where is it implemented? (JPQL in `StockBatchRepository`).
- 5. How do you calculate GST? Why `gst - cgst` for SGST?
- 6. How does login work end to end? (BCrypt -> JWT -> filter).
- 7. How is STAFF restricted from deleting products? (URL rules in `SecurityConfig`).
- 8. What does `@Transactional` do here? What happens on insufficient stock?
- 9. How do you avoid the N+1 problem? (JOIN FETCH + LAZY + open-in-view=false).
- 10. Why BigDecimal for money? What rounding do you use?
- 11. How does optimistic locking protect stock? (@Version -> 409).
- 12. How do you handle errors? (ApiException + @RestControllerAdvice, {status,message}).
- 13. How would two users billing at once behave?
- 14. What are the weaknesses of your invoice numbering? How would you fix it?
- 15. What would you do next to put this in production?
- 16. How do you keep one shop's data out of another shop's hands? (owner_id in every query; cross-tenant ids return 404.)

Top 10 things never to say

- 1. "I copied it from a tutorial." (own the decisions.)
- 2. "Redux/Kafka/Redis is used here." (they are not - the stack table is real.)
- 3. "The database is fully indexed." (no index DDL exists - say what you WOULD index.)
- 4. "JWT is 100% secure / tokens cannot be stolen." (acknowledge XSS risk of localStorage.)
- 5. "My code has no bugs." (name the CSS modal bug or the smoke-script issue instead.)
- 6. "I worked in a big team of 10." (it is solo - own it: "I owned everything, which taught me ...").
- 7. "Spring Security handles everything so authorization is automatic." (I wrote the URL rules.)
- 8. "H2 is production-ready." (it is the dev default; MySQL is the production path.)
- 9. "I don't know" with nothing after it. (Use: "I have not used it; my understanding is X, and I would verify by Y.")
- 10. Long silent pauses on numbers - rehearse: 34 tests, 62 checks, 20 products, 9 entities, 24 h token.

> **Remember:** On the interview day: 10 minutes this cheat sheet, then close the laptop. Confidence comes from the five flows, not from re-reading code.